1 MCP服务器开发

1.1 摘要

- mcp官方文档

> https://modelcontextprotocol.io/introduction

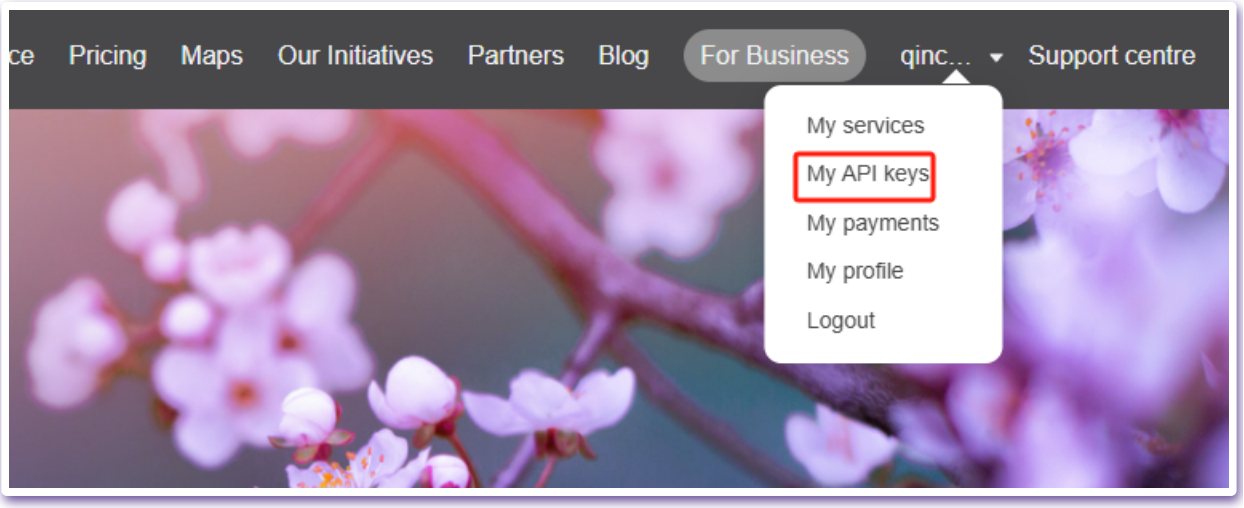
- openweather(国内可不用翻墙注册访问)

> https://openweathermap.org/

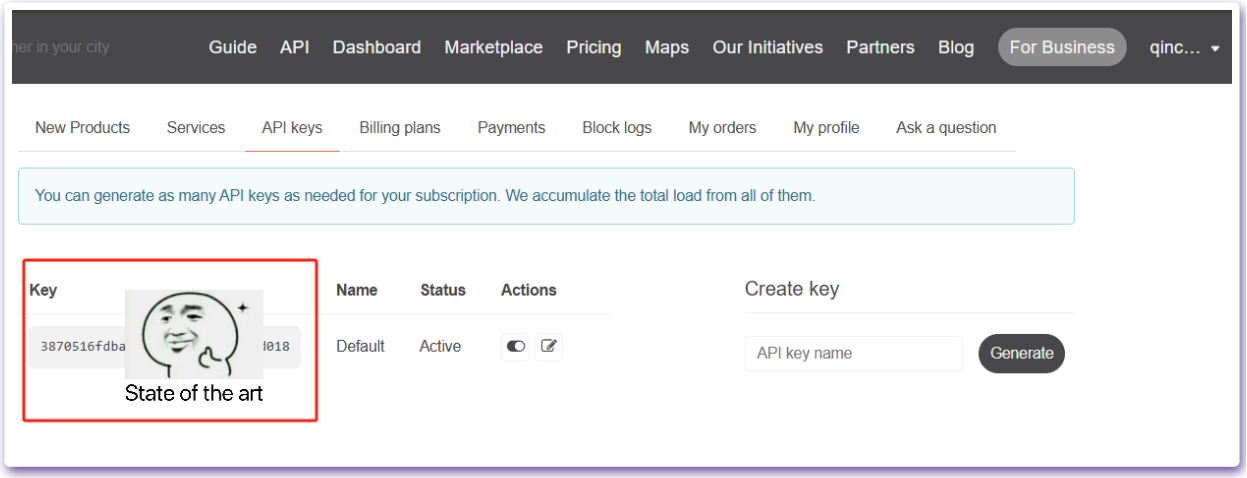
- 根据官方文档改编，主要区别如下
    1. 官网中用的是美国政府的天气网站，改成了openweather
    2. 官方文档中注册的两个mcp工具分别是用来"天气预报"和"天气预警"的，并且官网中的天气网站需要提供经纬度，而openweather是提供地址查询的，两个网站返回值不一致，但是openweather是会返回经纬度的，所以受官网启发，代码中注册的两个mcp工具分别变成了“当前天气查询”和“查询经纬度”
    3. 官网中使用的是Claude Desktop,国内使用账号非常容易被封，所以使用支持mcp服务器的app，cherrystudio来演示

1.2 openweather的使用

- 打开 openweather 的地址,注册完成后点击右上角自己的昵称



找到自己的api-key



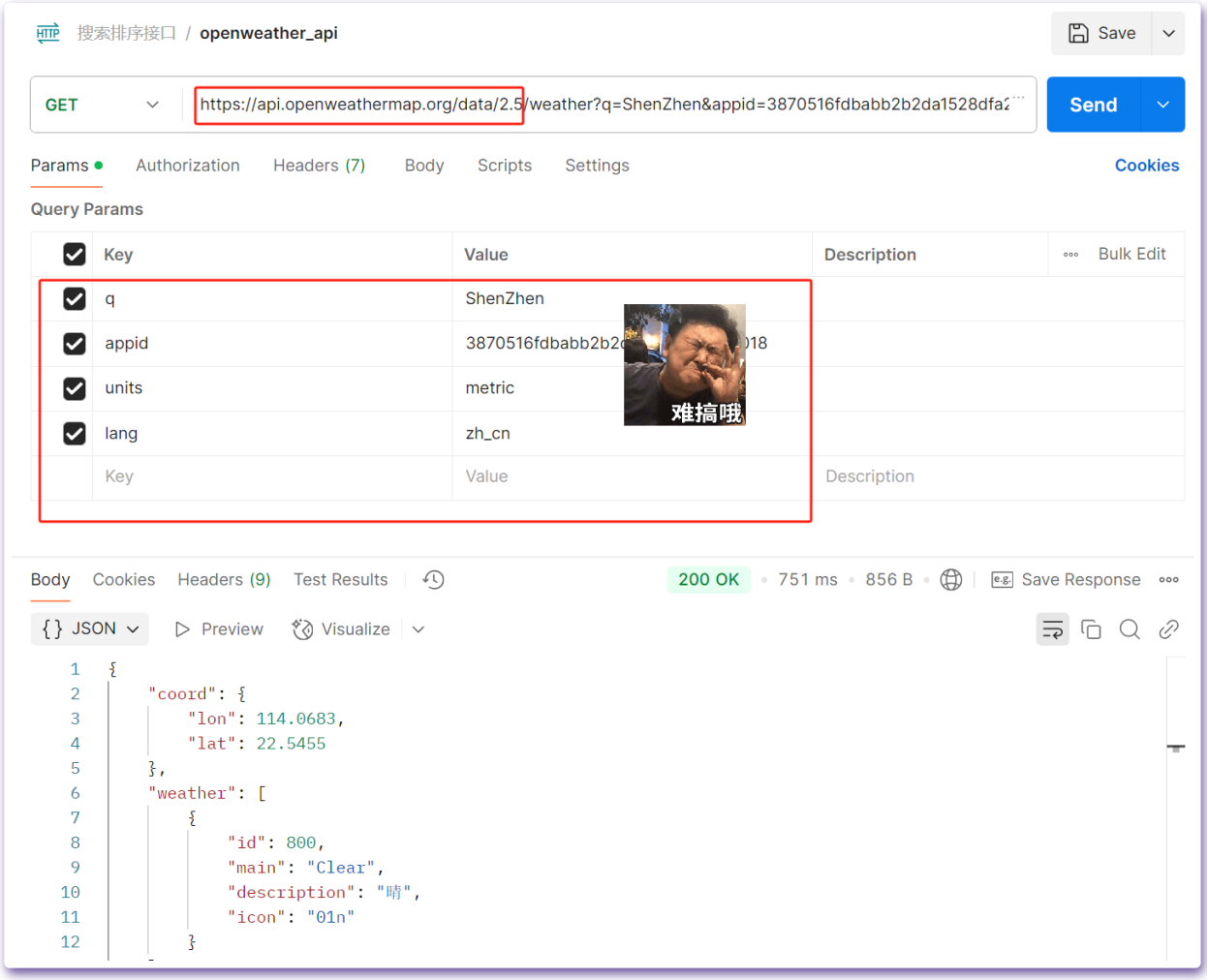
- 调用示例

  调用方式：GET

  调用地址：https://api.openweathermap.org/data/2.5/weather

  curl命令如下

```
curl --location 'https://api.openweathermap.org/data/2.5/weather?q=地址&appid=自己的api-key&units=metric&lang=zh_cn'
```



1.3 服务器开发

1.3.1 环境搭建

- 官方推荐使用uv来管理环境，先下载uv

```
pip install uv

#或者macos linux
curl -LsSf https://astral.sh/uv/install.sh | sh

#或者windows
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

  之后重启终端

- 初始化项目。名称为weather

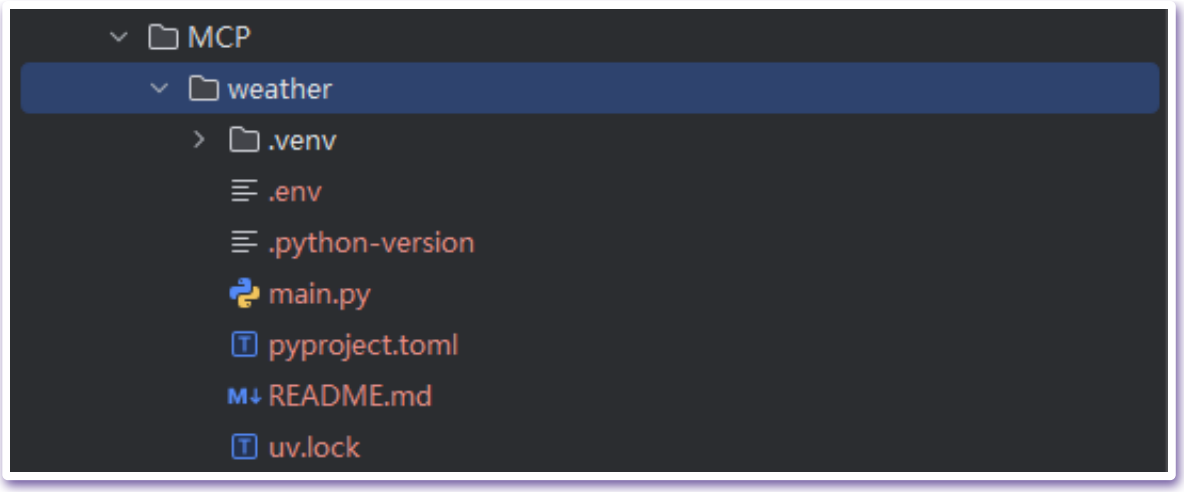
```
uv init weather
```

- 创建虚拟环境并激活

```
cd weather
uv venv

# windows
.venv\Scripts\activate

# linux
source .venv/bin/activate
```

- 安装依赖

```
# windows
uv add mcp[cli] httpx dotenv

# linux
uv add "mcp[cli]" httpx dotenv
```



1.3.2 构建服务器

- 在weather目录创建环境变量文件.env

```
OPENWEATHER_BASE_URL=https://api.openweathermap.org/data/2.5/weather
OPENWEATHER_API_KEY=你的api-key
```

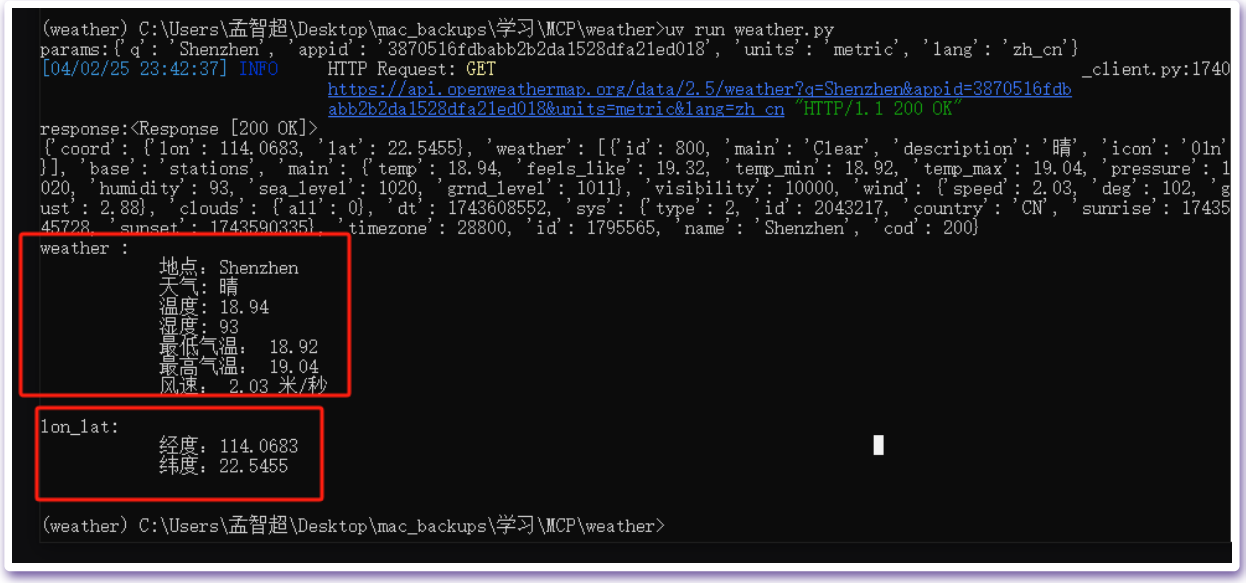
- weather.py

```python
# -*- coding: utf-8 -*-
"""
@author: qinchihongye
@contact: qinchihongye@gmail.com
@created: 2025/4/2
@version: 1.0

This script does something useful.
"""
from typing import Any
import httpx
from mcp.server.fastmcp import FastMCP
import os
from dotenv import load_dotenv
import asyncio

load_dotenv() #加载环境变量

# Initialize FastMCP server 初始化 mcp 服务器
mcp = FastMCP("weather")

# Constants 加载环境变量
openweather_base_url = os.getenv("OPENWEATHER_BASE_URL")
openweather_api_key = os.getenv("OPENWEATHER_API_KEY")
USER_AGENT = "weather-app/1.0"

# 辅助函数-查询天气
async def make_nws_request(location: str):
```

```python
    """查询天气"""
    headers = {
        "User-Agent": USER_AGENT,
        "Accept": "application/geo+json"
    }
    params = {
        "q": location,  # 地址
        "appid": openweather_api_key,
        "units": "metric",
        "lang": "zh_cn"

    }
    print(f'params:{params}')
    async with httpx.AsyncClient() as client:
        try:
            response = await client.get(openweather_base_url, headers=headers, params=params, timeout=30.0)
            print(f'response:{response}')
            response.raise_for_status()
            return response.json()
        except Exception:
            return None

# 辅助函数-格式化天气数据并返回
async def format_weather(feature: dict) -> str:
    """格式化天气数据并返回"""
    props = feature["main"]
    return f"""
            地点: {feature.get("name", "未知")}
            天气: {feature.get("weather", {})[0]['description']}
            温度: {props.get("temp", "未知")}
            湿度: {props.get("humidity", "未知")}
            最低气温:  {props.get("temp_min", "未知")}
            最高气温:  {props.get("temp_max", "未知")}
            风速:  {feature.get("wind", {}).get("speed", "未知")}米/秒
            """

# 辅助函数-从天气信息中提取经纬度
async def format_lon_lat(feature: dict) -> str:
    """从天气信息中提取经纬度"""
    props = feature["coord"]
    return f"""
            经度: {props.get("lon", "未知")}
            纬度: {props.get("lat", "未知")}
            """


@mcp.tool()
async def get_weather(location: str):
    """获取当前地点当前时间的天气信息"""
```

```python
    weather_json = await make_nws_request(location)  # 根据给定地点请求天气信息
    formatted_weather = await format_weather(weather_json) # 格式化天气数据并返回

    return formatted_weather

@mcp.tool()
async def get_lon_lat(location:str):
    """获取当前地点的经纬度信息"""
    weather_json = await make_nws_request(location) # 根据给定地点请求天气信息
    formatter_lon_lat = await format_lon_lat(weather_json) # 获取经纬度信息并返回
    return formatter_lon_lat

async def test():
    """仅用于测试"""

    result = await make_nws_request('深圳')
    print(result)
    weather = await format_weather(result)
    print(f"weather : {weather}")
    lon_lat = await format_lon_lat(result)
    print(f"lon_lat: {lon_lat}")


if __name__ == "__main__":
    # asyncio.run(test()) # 仅用于测试
    mcp.run(transport='stdio')
```

- 主要代码解析
    1. `make_nws_request` 主要通过输入地点（中国的地点要是拼音），查询天气信息并返回
    2. `format_weather` 直接对 `make_nws_request` 的输出进行解析，格式化天气数据并返回
    3. `format_lon_lat` 直接对 `make_nws_request` 的输出进行解析，但是仅仅提取经纬度并返回
    4. `get_weather` 为需要注册的查询天气信息的服务端工具
    5. `get_lon_lat` 为需要注册的查询经纬度信息的服务端工具
    6. `test` 为测试辅助函数的代码
- test函数测试如下



- 运行uv run weather.py以确认一切正常

1.4 app端使用

1.4.1 安装cherrystudio

- 安装cherrystudio,并配置好模型

> https://cherry-ai.com/

1.4.2 app端添加服务器

- 在app端添加单个weather服务器,下面的json文件需要复制到cherrystudio中

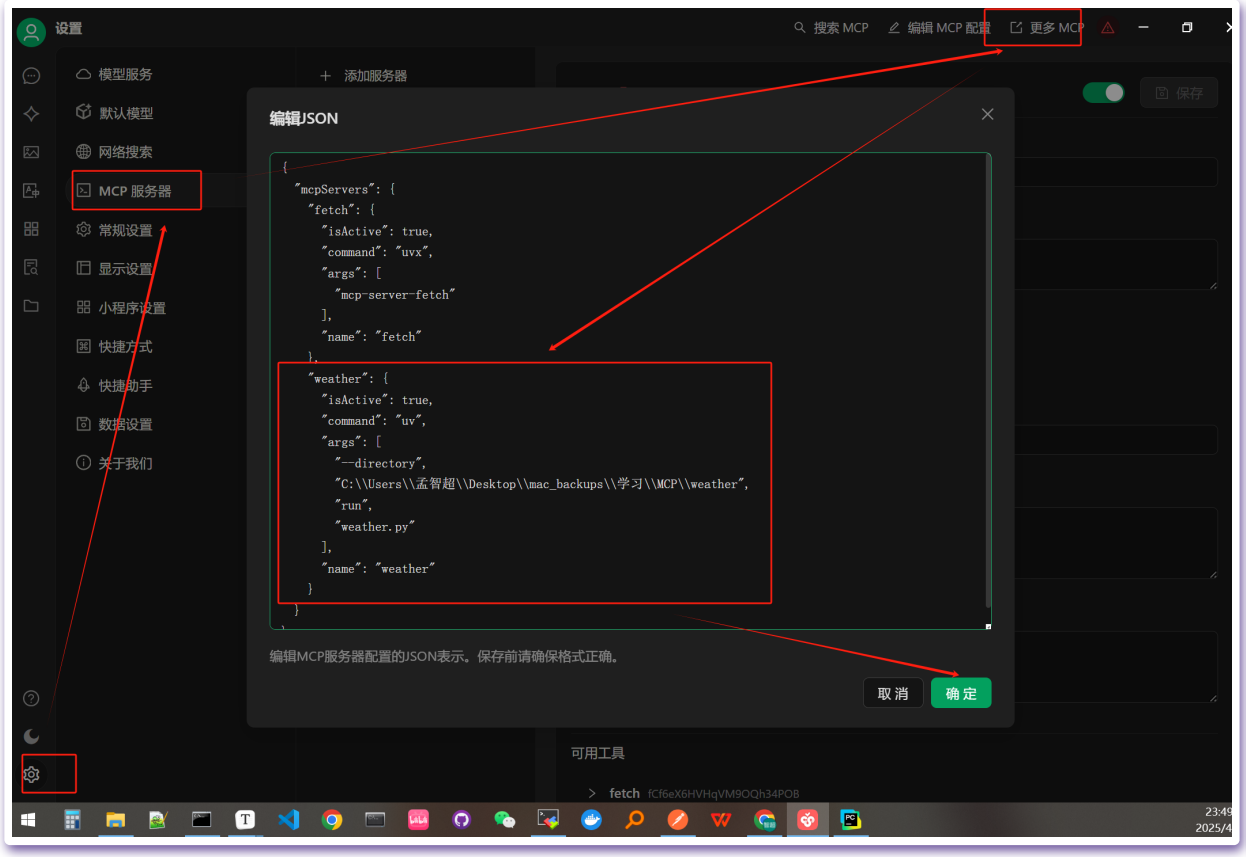
```
{
    "mcpServers": {
        "weather": {
            "command": "uv",
            "args": [
                "--directory",
                "C:\\Users\孟智超\\Desktop\\mac_backups\\学习\\MCP\\weather",
                "run",
                "weather.py"
            ]
        }
    }
}
```

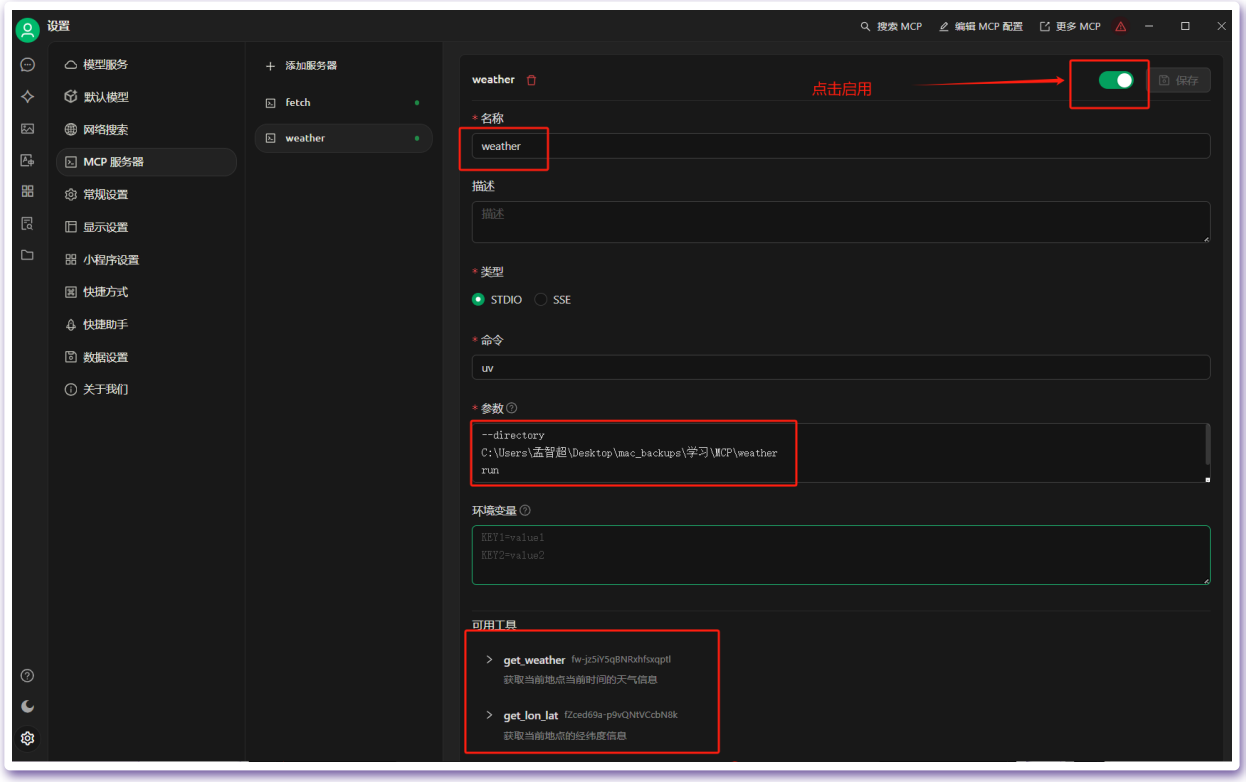
意思是:

1. 有一个名为“天气”的 MCP 服务器
2. 通过运行 `uv --directory C:\\Users\孟智超\\Desktop\\mac_backups\\学习`

`\\MCP\\weather run weather.py` 来启动它（项目的地址）

- 将上面的json文本复制到cherrystudio



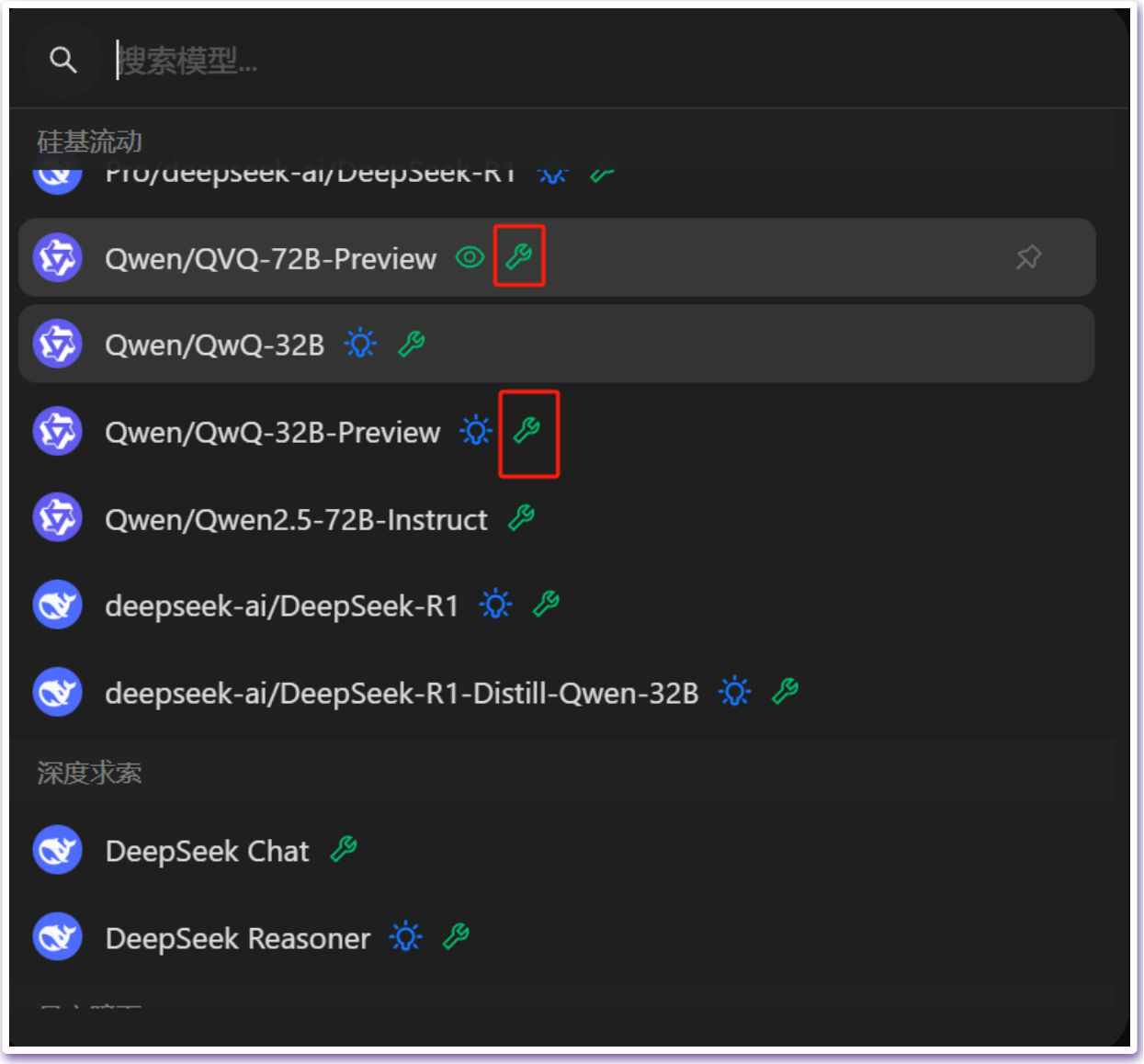


点击启用，下面的都是自动识别到的，包括工具也是自动识别的

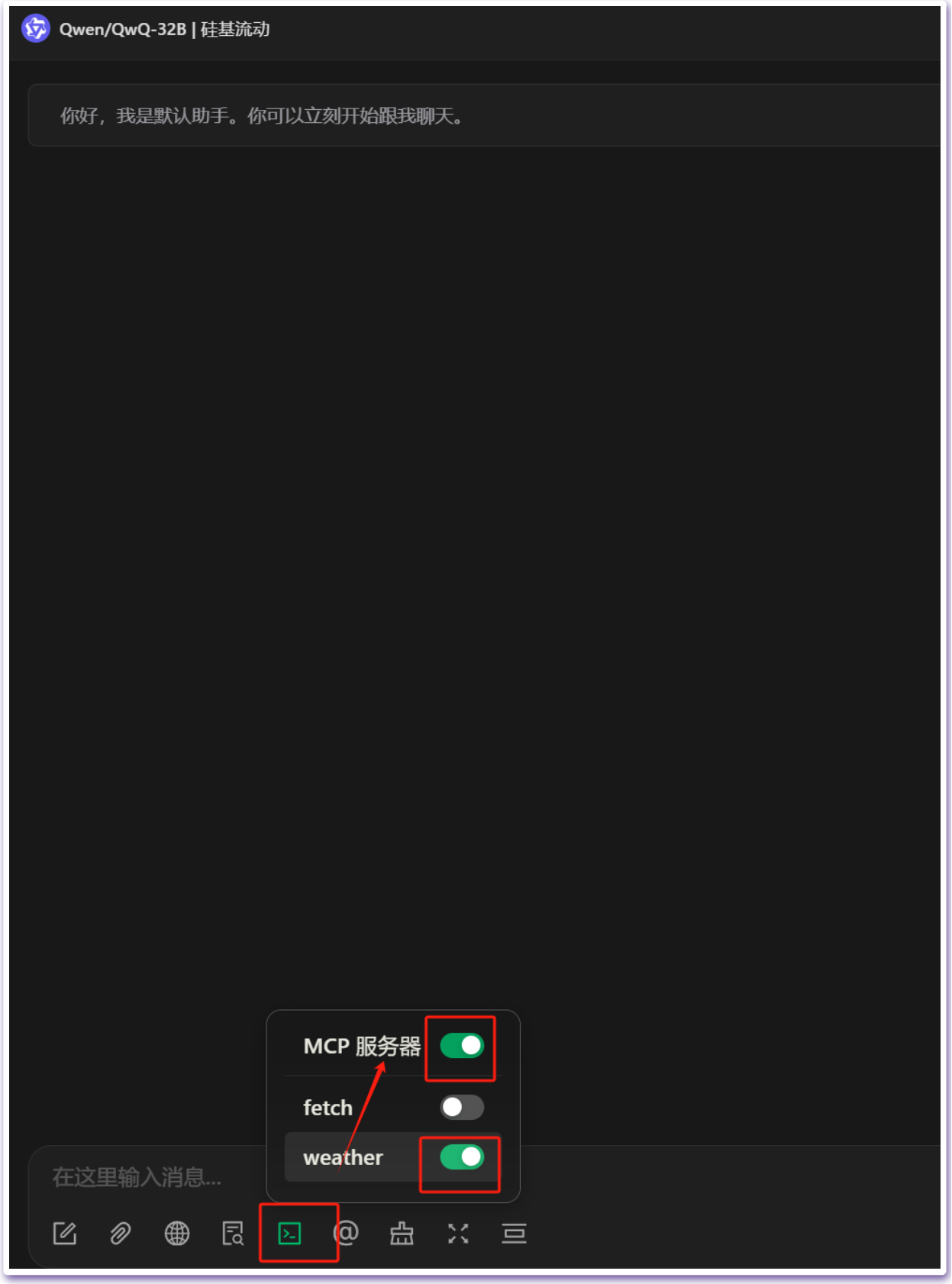
1.4.3 使用

- 选择模型，因为mcp也借用了functioncalling，所以需要选用的模型必须支持functioncalling（tools）

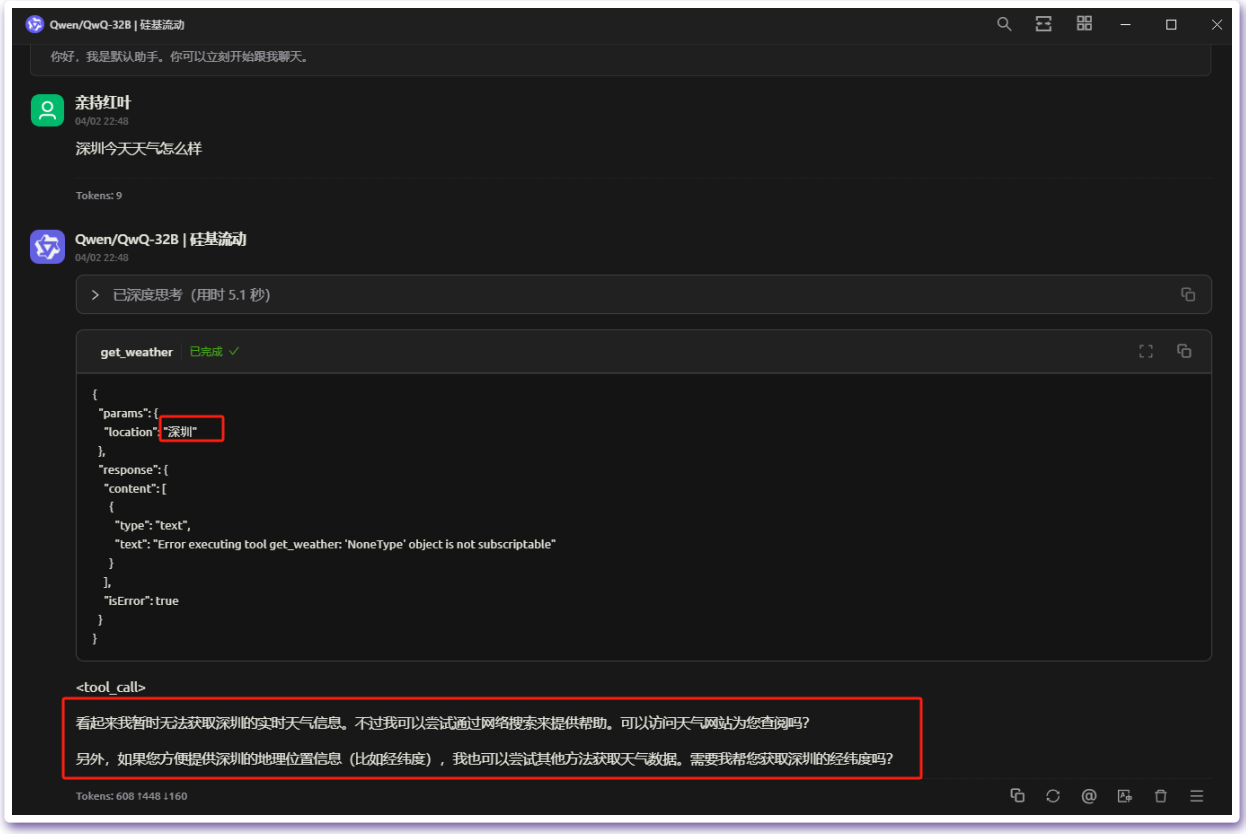
  在cherrystudio中的表现就是，有绿色的小扳手



- 开始聊天之前开启mcp



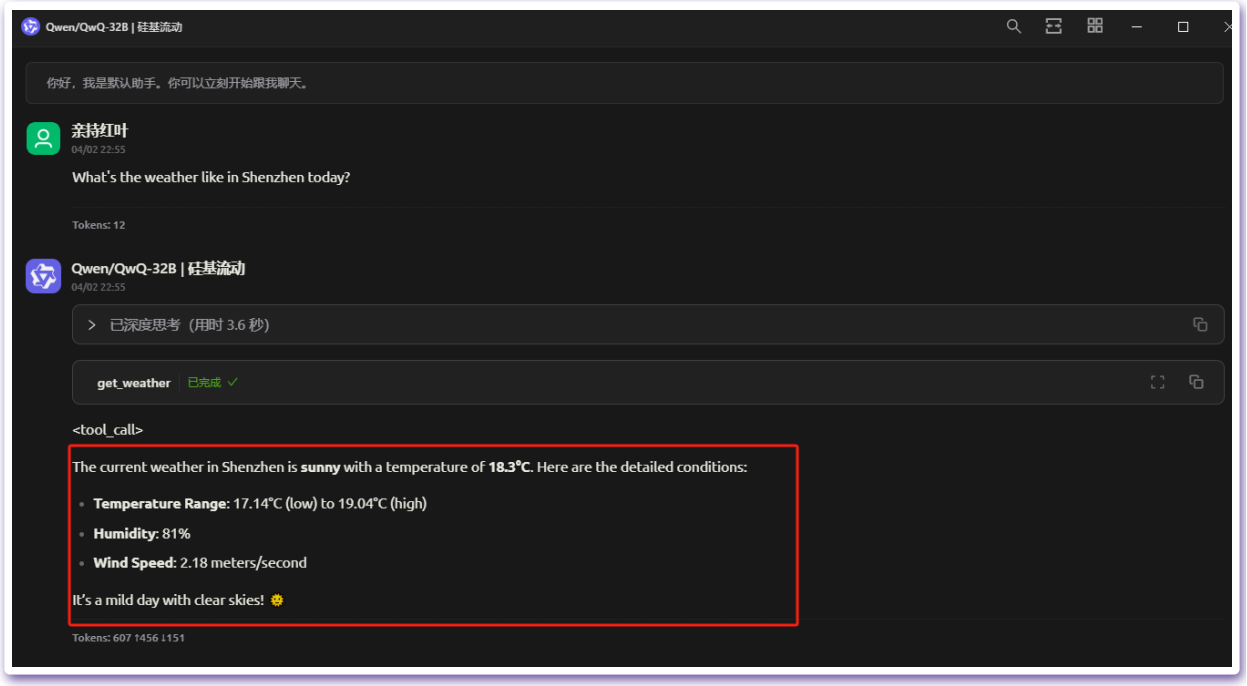
- 首次调用失败了，原因是我们使用了中文提问，然后模型传入的地址是中文的，但是openweather要求的是传入中文拼音



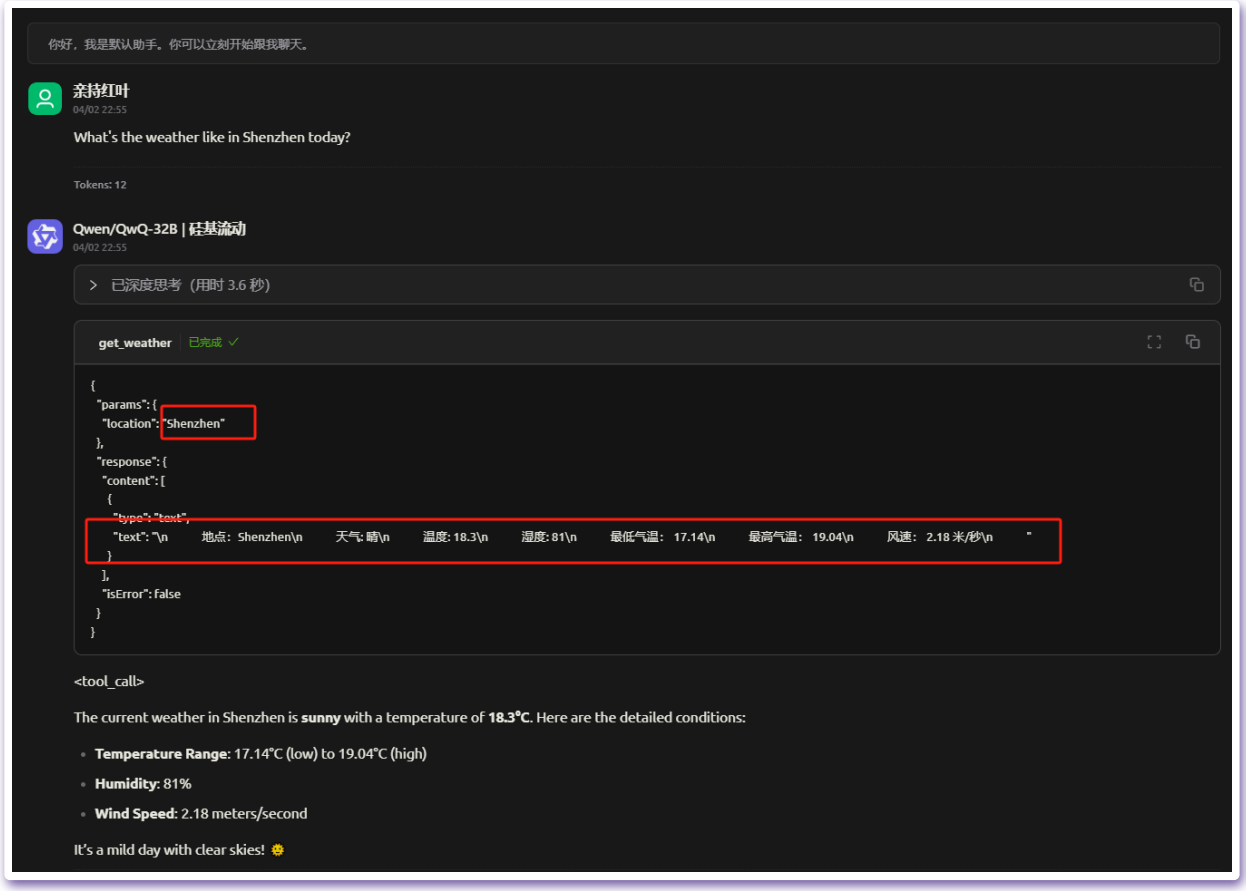
- 那我们尝试使用英语提问

  先问一下深圳今天的天气怎么样

  What's the weather like in Shenzhen today?



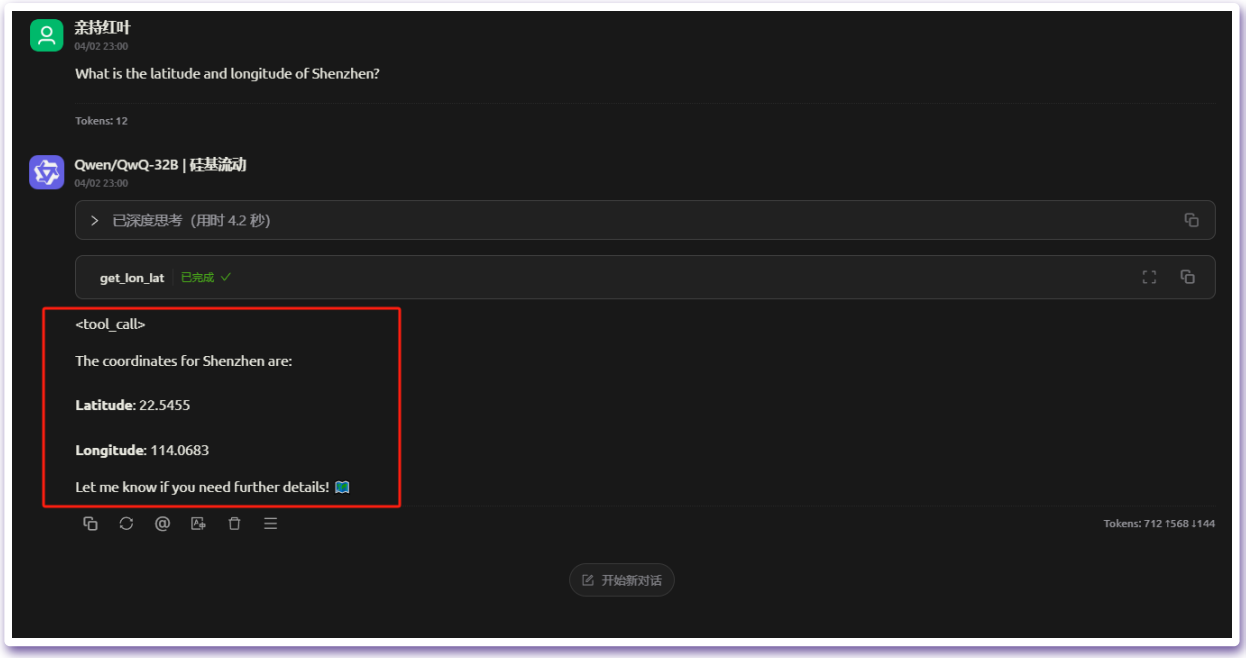
这次成功了，我们看看get_weather给我们返回了什么



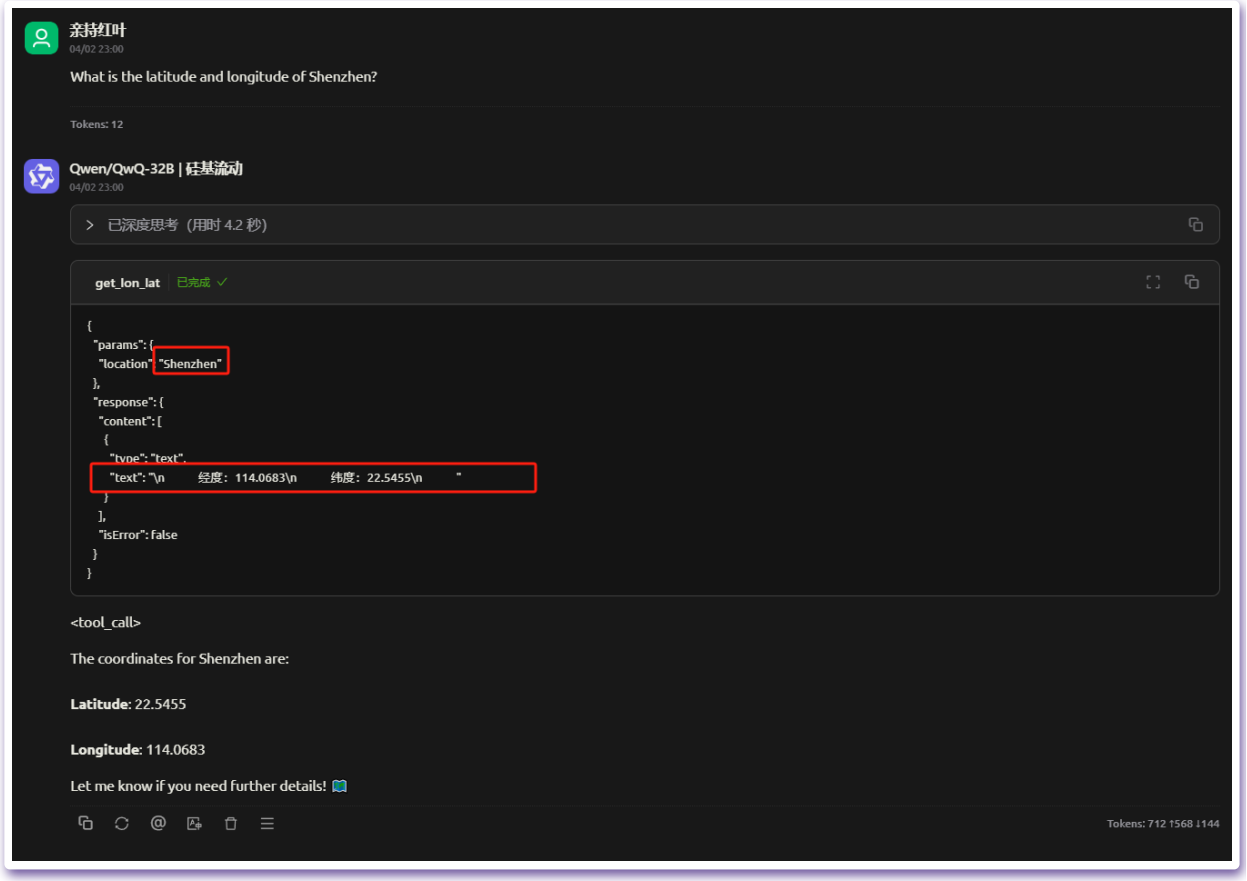
- 再看看经纬度

  深圳的经纬度是多少

  What is the latitude and longitude of Shenzhen?



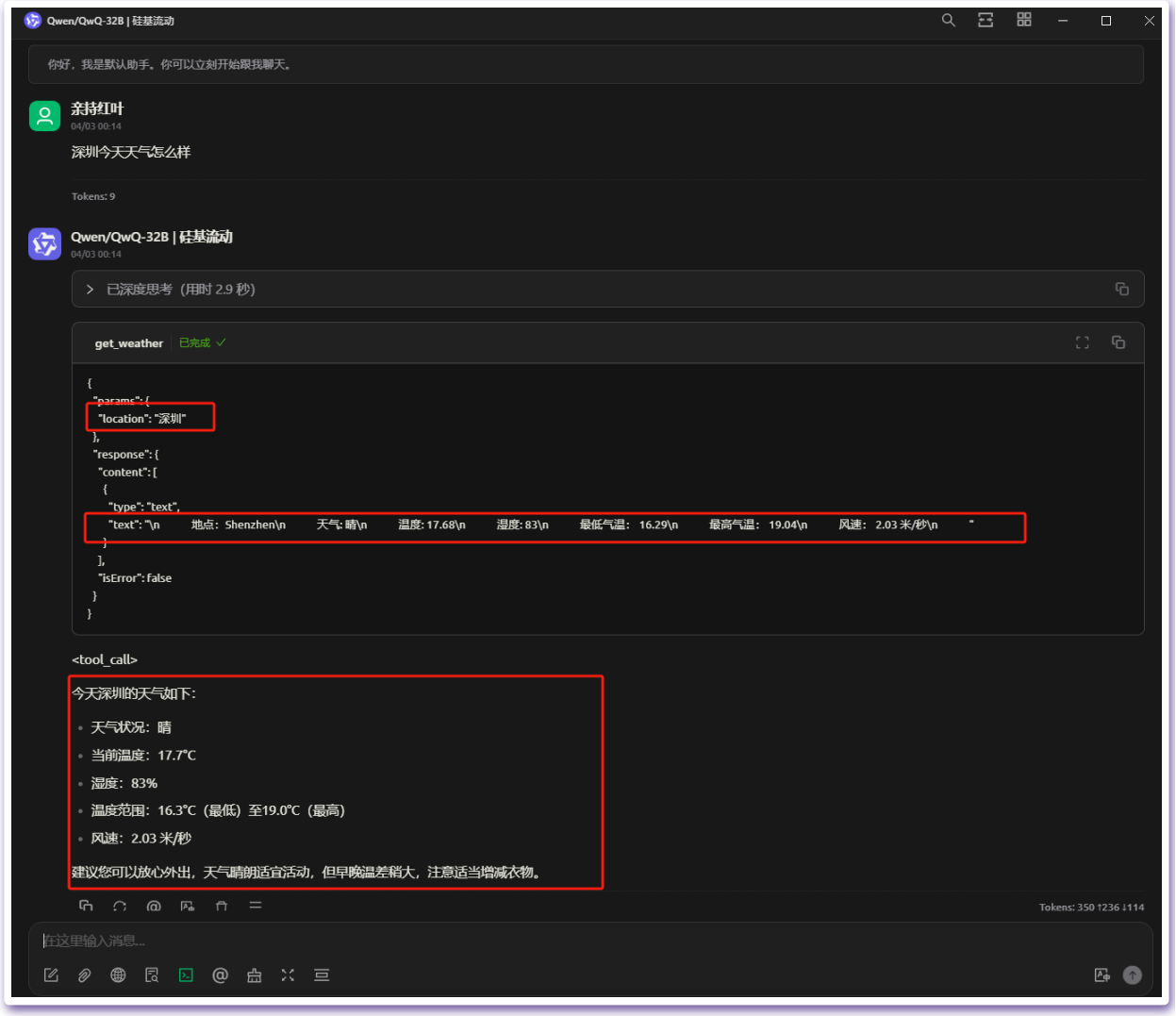
ok,也没有问题，再看看get-lon_lat返回的是什么



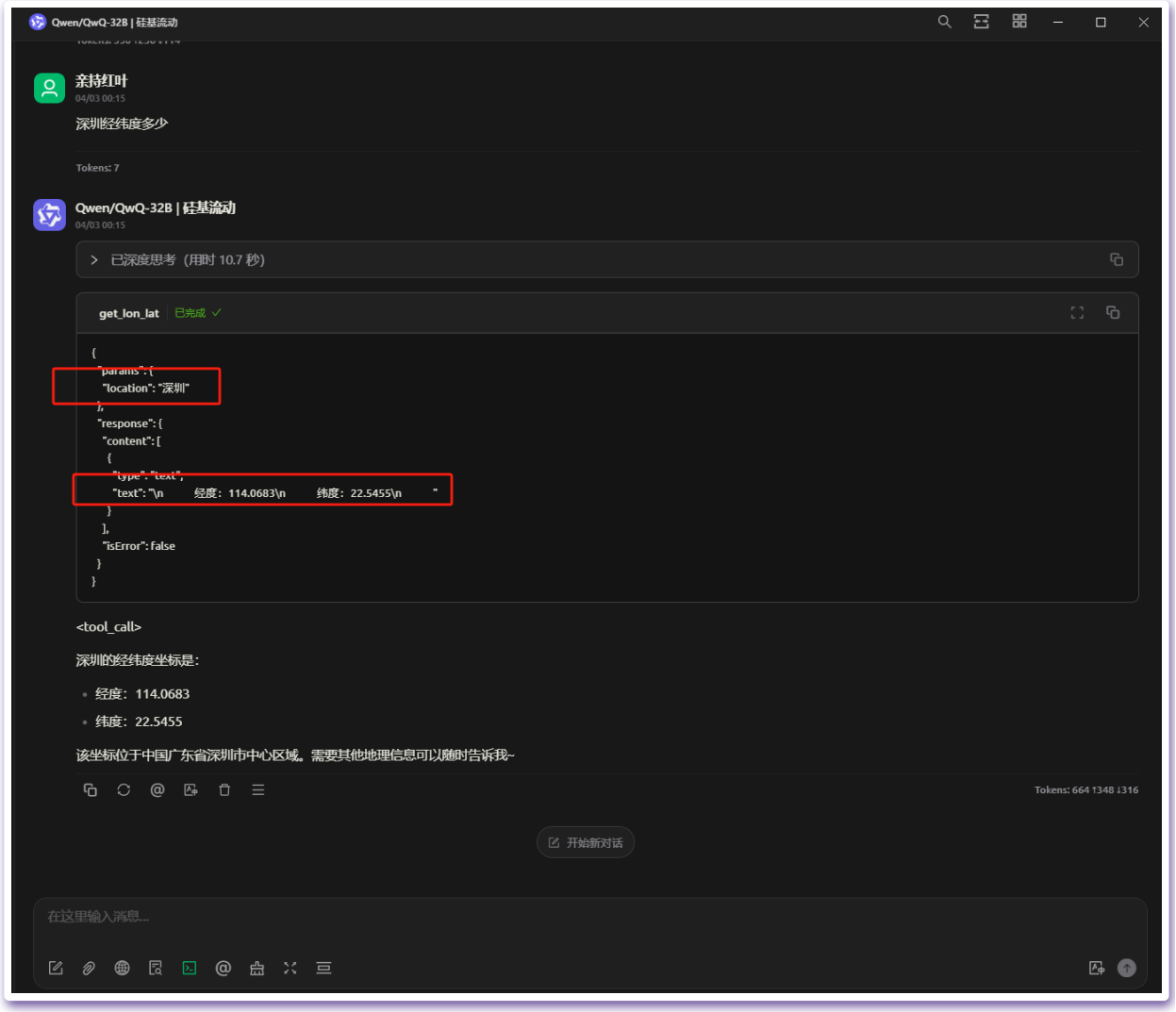
- 后续在weather.py中加上如果location输入为中文则转为拼音的代码,之后再次重试

```python
from pypinyin import lazy_pinyin
import re

def convert_to_pinyin(input_string):
    # 检查字符串是否包含中文字符
    if re.search(r'[\u4e00-\u9fff]', input_string):
        # 如果包含中文字符，转换为拼音
        pinyin_list = lazy_pinyin(input_string)
        return ''.join(pinyin_list)
    else:
        # 如果不包含中文字符，返回原字符串
        return input_string
```

先查看天气


Qwen/QwQ-32B | 硅基流动
你好，我是默认助手。你可以立刻开始跟我聊天。
亲持红叶
04/03 00:14
深圳今天天气怎么样
Tokens: 9
Qwen/QwQ-32B | 硅基流动
04/03 00:14
已深度思考（用时 2.9 秒）
get_weather
已完成
{
"params": {
"location": "深圳"
},
"response": {
"content": [
{
"type": "text",
"text": "\n 地点：Shenzhen\n 天气: 晴\n 温度: 17.68\n 湿度: 83\n 最低气温：16.29\n 最高气温：19.04\n 风速：2.03 米/秒\n "
}
],
"isError": false
}
}
<tool_call>
今天深圳的天气如下：
天气状况：晴
当前温度：17.7°C
湿度：83%
温度范围：16.3°C（最低）至19.0°C（最高）
风速：2.03 米/秒
建议您可以放心外出，天气晴朗适宜活动，但早晚温差稍大，注意适当增减衣物。
Tokens: 350 ↑236 ↓114
在这里输入消息...

ok,再看看经纬度



ok

1.4.4 最终代码

- 最终代码

```
# -*- coding: utf-8 -*-
"""
@author: qinchihongye
@contact: qinchihongye@gmail.com
@created: 2025/4/2
@version: 1.0

This script does something useful.
"""
from typing import Any
import httpx
from mcp.server.fastmcp import FastMCP
import os
from dotenv import load_dotenv
import asyncio
from pypinyin import lazy_pinyin
import re

```

```python

load_dotenv() #加载环境变量

# Initialize FastMCP server 初始化 mcp 服务器
mcp = FastMCP("weather")

# Constants 加载环境变量
openweather_base_url = os.getenv("OPENWEATHER_BASE_URL")
openweather_api_key = os.getenv("OPENWEATHER_API_KEY")
USER_AGENT = "weather-app/1.0"

# 辅助函数-查询天气
async def make_nws_request(location: str):
    """查询天气"""
    headers = {
        "User-Agent": USER_AGENT,
        "Accept": "application/geo+json"
    }
    params = {
        "q": location,  # 地址
        "appid": openweather_api_key,
        "units": "metric",
        "lang": "zh_cn"

    }
    print(f'params:{params}')
    async with httpx.AsyncClient() as client:
        try:
            response = await client.get(openweather_base_url, headers=headers, params=params, timeout=30.0)
            print(f'response:{response}')
            response.raise_for_status()
            return response.json()
        except Exception:
            return None

# 辅助函数-格式化天气数据并返回
async def format_weather(feature: dict) -> str:
    """格式化天气数据并返回"""
    props = feature["main"]
    return f"""
            地点: {feature.get("name", "未知")}
            天气: {feature.get("weather", {})[0]['description']}
            温度: {props.get("temp", "未知")}
            湿度: {props.get("humidity", "未知")}
            最低气温:  {props.get("temp_min", "未知")}
            最高气温:  {props.get("temp_max", "未知")}
            风速:  {feature.get("wind", {}).get("speed", "未知")}米/秒
            """

# 辅助函数-从天气信息中提取经纬度
```

```python
async def format_lon_lat(feature: dict) -> str:
    """从天气信息中提取经纬度"""
    props = feature["coord"]
    return f"""
            经度: {props.get("lon", "未知")}
            纬度: {props.get("lat", "未知")}
            """

async def convert_to_pinyin(input_string):
    # 检查字符串是否包含中文字符
    if re.search(r'[\u4e00-\u9fff]', input_string):
        # 如果包含中文字符，转换为拼音
        pinyin_list = lazy_pinyin(input_string)
        return ''.join(pinyin_list)
    else:
        # 如果不包含中文字符，返回原字符串
        return input_string

@mcp.tool()
async def get_weather(location: str):
    """获取当前地点当前时间的天气信息"""
    location = await convert_to_pinyin(location) # 如果是中文则转换为拼音
    weather_json = await make_nws_request(location)  # 根据给定地点请求天气信息
    formatted_weather = await format_weather(weather_json) # 格式化天气数据并返回

    return formatted_weather

@mcp.tool()
async def get_lon_lat(location:str):
    """获取当前地点的经纬度信息"""
    location = await convert_to_pinyin(location) # 如果是中文则转换为拼音
    weather_json = await make_nws_request(location) # 根据给定地点请求天气信息
    formatter_lon_lat = await format_lon_lat(weather_json) # 获取经纬度信息并返回
    return formatter_lon_lat

async def test():
    """仅用于测试"""
    location = "深圳"
    location = await convert_to_pinyin(location)
    print(f'location: {location}')
    result = await make_nws_request(location)
    print(result)
    weather = await format_weather(result)
    print(f"weather : {weather}")
    lon_lat = await format_lon_lat(result)
    print(f"lon_lat: {lon_lat}")
```

```python


if __name__ == "__main__":
    # asyncio.run(test()) # 仅用于测试
    mcp.run(transport='stdio')
```

2 MCP项目开发-一个简单的RAG示例

2.1 前言

2.1.1 前言

- 客户端是基于官网的例子改的，模型改成了openai库连接
- 仅仅使用基础的RAG流程作为一个演示，包含了以下步骤
    1. query改写
    2. 搜索：使用google serper
    3. 重排序：使用硅基流动的api
- 大模型api也使用的硅基流动
- 调用速度还可以
- **Hosts** are LLM applications (like Claude Desktop or IDEs) that initiate connections
  主机是发起连接的应用（例如 Claude Desktop 或 IDE）

  Clients maintain 1:1 connections with servers, inside the host application
  客户端在主机应用程序内部与服务器保持 1：1 连接

  Servers provide context, tools, and prompts to clients
  服务器向客户提供上下文、工具和提示

2.1.2 官方文档

- mcp官方文档

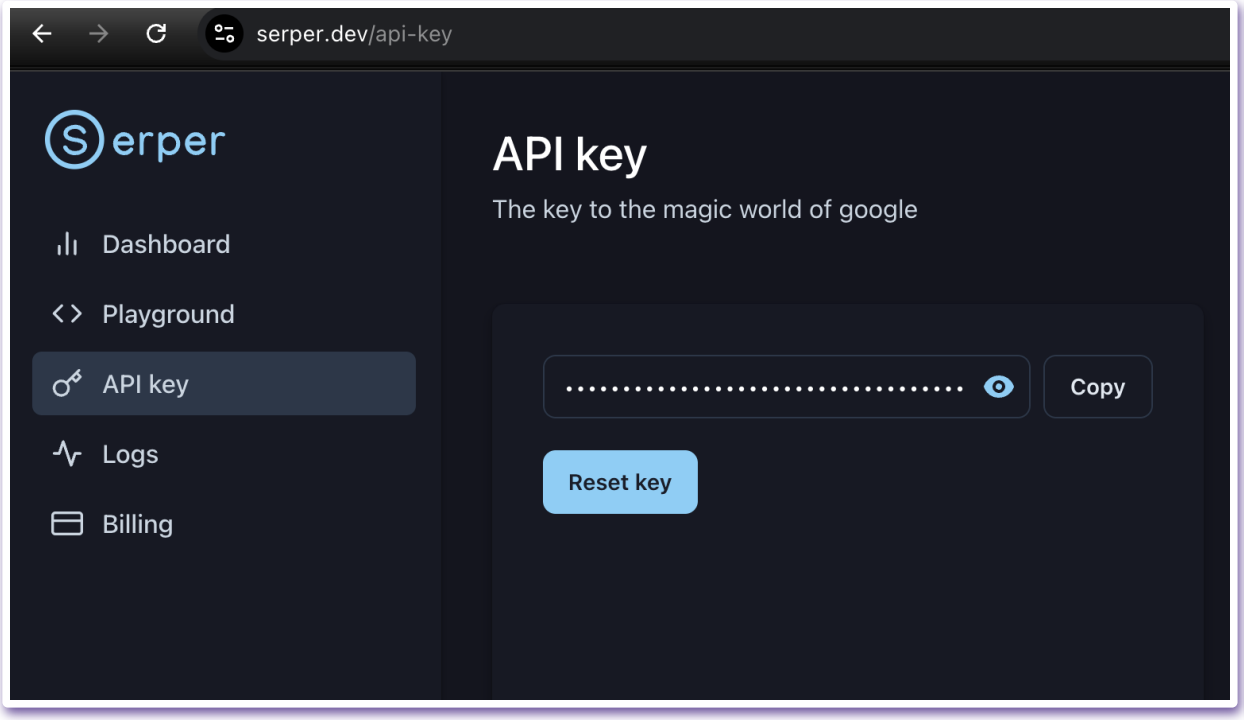
> https://modelcontextprotocol.io/introduction

2.1.3 使用到的api注册

- 检索：google serper

> https://serper.dev

有2500 Credits，具体也没搞明白，好像是每次检索返回的文本数量超过10则花费2Credits，不过这个不重要，反正自己用肯定够,api在这



连接示例

```
curl --location 'https://google.serper.dev/search' \
--header 'X-API-KEY: 你的api-key' \
--header 'Content-Type: application/json' \
--data '{
    "q": "你的问题",
    "num": 2
}'
```

其中q表示要检索的问题，然后num为返回的文本数,返回示例如下

```
{
    "searchParameters": {
        "q": "中国首都是哪",
        "type": "search",
        "num": 2,
        "engine": "google"
    },
    "organic": [
        {
            "title": "中华人民共和国首都 - 维基百科",
            "link": "https://zh.wikipedia.org/zh-hans/%E4%B8%AD%E5%8D%8E%E4%BA%BA%E6%B0%91%E5%85%B1%E5%92%8C%E5%9B%BD%E9%A6%96%E9%83%BD",
            "snippet": "中华人民共和国首都位于北京市，新中国成立前夕的旧称为北平，是中共中央及中央人民政府所在地，中央四个直辖市之一，全国政治、文化、国际交往和科技创新中心，中国古都、 ...",
            "position": 1
        },
        {
            "title": "中华人民共和国首都_中华人民共和国中央人民政府门户网站",
```

```
            "link": "https://www.gov.cn/guoqing/2005-05/24/content_2615214.htm",
            "snippet": "1949年9月27日，中国人民政治协商会议第一届全体会议一致通过中华人民共和国的国都定于北平，即日起北平改名北京。 北京，简称京，是中国共产党中央委员会、中华人民共和国 ...",
            "position": 2
        }
    ],
    "relatedSearches": [
        {
            "query": "中国首都是北京还是上海"
        },
        {
            "query": "中国首都为什么是北京"
        },
        {
            "query": "中国首都上海"
        },
        {
            "query": "中国以前的首都"
        },
        {
            "query": "新中国首都选址"
        },
        {
            "query": "中国历代首都"
        },
        {
            "query": "中国首都候选"
        },
        {
            "query": "中国首都英文"
        }
    ],
    "credits": 1
}
```


主要的就是在 `organic` 中，如果我们的num为2，则organic中返回两个dict，我们取的是没个dict中的snippet字段

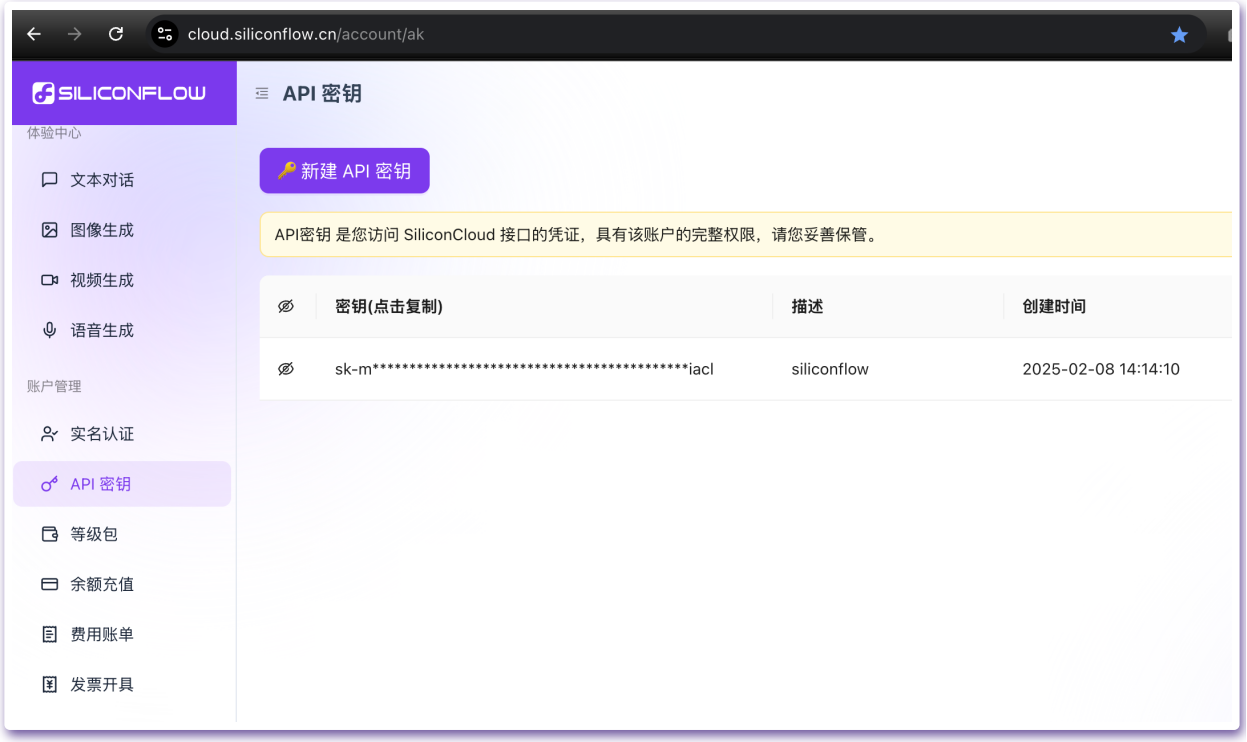
- 聊天大模型与重排序模型：硅基流动

> **https://cloud.siliconflow.cn/**

注册可以看我的另一篇

> **https://blog.csdn.net/hbkybkzw/article/details/145558234**

api-key如图



2.2 服务器

2.2.1 环境搭建

- 官方推荐使用uv来管理环境，先下载uv

```
pip install uv

#或者macos linux
curl -LsSf https://astral.sh/uv/install.sh | sh

#或者windows
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

  之后重启终端

- 初始化项目。名称为rag_sample

```
uv init rag_sample
```

- 创建虚拟环境并激活

```
cd rag_sample
uv venv

# windows
.venv\Scripts\activate

# linux
source .venv/bin/activate
```

- 安装依赖

  openai库是客户端用的，为了方便，客户端就不初始化了，直接和服务器用同一个环境

```
# windows
uv add mcp[cli] aiohttp dotenv openai rich

# linux
uv add "mcp[cli]" aiohttp dotenv openai rich
```

2.2.2 构建服务器

- 在rag_sample目录下，创建 .env 文件保存为环境变量

```
GOOGLE_SERPER_URL=https://google.serper.dev/search
GOOGLE_SERPER_API=
SILICONFLOW_RERANKING_BASE_URL=https://api.siliconflow.cn/v1/rerank
SILICONFLOW_CHAT_BASE_URL=https://api.siliconflow.cn/v1/chat/completions
SILICONFLOW_RERANKING_API_KEY=
RERANKING_MODEL=BAAI/bge-reranker-v2-m3
OPENAI_CHAT_MODEL=Qwen/QwQ-32B
OPENAI_BASE_URL=https://api.siliconflow.cn/v1
```

  将 `GOOGLE_SERPER_API` 和 `SILICONFLOW_RERANKING_API_KEY` 填为自己的

- `rag_client.py`

```
import mcp
import aiohttp
import asyncio
from mcp.server.fastmcp import FastMCP
from utils import query_rewrite,search_with_serper,reranking_by_bge

mcp = FastMCP("rag")


@mcp.tool()
async def rag_sample(query:str,doc_num:int=10,top_n:int=4) -> list:
    """rag_sample:   输入query,输出检索重排序后的文档

    Parameters:
```

```
    ------------------------------------------------------------------------------------------------------------
    query:                type: str               desc: 用户问题

    doc_num:              type: int               desc: 检索文档数

    top_n:                type: top_n             desc: 重排序后返回top_n

    Returns:
    ------------------------------------------------------------------------------------------------------------
    reranked_doc:         type: list              desc: top_n检索文档

    """
    rewrite_query = await query_rewrite(query=query)
    documents = await search_with_serper(query=rewrite_query,document_num=doc_num)
    reranked_doc = await reranking_by_bge(query=query,documents=documents,top_n=top_n)
    return reranked_doc


async def test():
    query = "MCP协议介绍"
    reranked_doc = await rag_sample(query=query)
    import
    rich.print(reranked_doc)


if __name__ == "__main__":
    # asyncio.run(test())
    mcp.run(transport="stdio")
```

test函数是用来测试的

```
(rag_sample) PS C:\Users\孟智超\Desktop\mac_backups\MCP\rag_sample> python .\rag_server.py
[
    'MCP协议是什么？MCP(Model Context Protocol, 模型上下文协议)
是一种新的开放标准协议，用来在大模型和数据源之间建立安全双向的链接。对照下面这个图，我们来理解下MCP。',
    'Model Context Protocol (MCP)
模型上下文协议，是由claude的发布公司Anthropic，提出的一种开放协议。为给LLM提供上下文（数据）提供了一种标准协议。',
    'MCP（Model Context Protocol，模型上下文协议）是由Anthropic 提出并于2024 年11
月开源的一种通信协议，旨在解决大型语言模型（LLM）与外部数据源及工具之间无 ...',
    'MCP 是一种新的开放标准协议，用来在大模型和数据源之间建立安全双向的链接。这是它的官方示意图。 这张图展示了使用Claude 或其他IDE
时，通过这种 ...'
]
(rag_sample) PS C:\Users\孟智超\Desktop\mac_backups\MCP\rag_sample>
```

我们将主要的代码放在 `rag_client.py`，辅助函数都放在 `utils.py` 中

主要的流程就是

1. `query_rewrite`： 传入用户问题，返回改写后的问题
2. `search_with_serper`： 传入改写后的问题和检索数量，返回检索到的文档信息

3. `reranking_by_bge`：传入用户问题和检索到的信息，返回相似度高的top_n文档信息

代码见下面的utils.py

- `utils.py`

```python
import asyncio
import aiohttp
from dotenv import load_dotenv
import os

#从.env文件加载环境变量
load_dotenv()

# google serper
google_serper_url = os.getenv("GOOGLE_SERPER_URL")
google_serper_api = os.getenv("GOOGLE_SERPER_API")

# siliconflow api
chat_base_url = os.getenv("SILICONFLOW_CHAT_BASE_URL")
reranking_base_url = os.getenv("SILICONFLOW_RERANKING_BASE_URL")
reranking_api_key = os.getenv("SILICONFLOW_RERANKING_API_KEY")
reranking_model = os.getenv("RERANKING_MODEL")

# query rewrite prompt
query_rewrite_prompt = """
请根据以下规则优化用户的问题：
1. 首先判断用户query是否为问候或打招呼（例如：'你好'、'在吗'、'您好'等）。如果是问候，直接返回原文query。
2. 如果不是，对query进行优化改写，请保持原意并使改写后的query更精准更简洁,限制在15个字以内
示例：
    - 输入：'你好，能帮个忙吗？'
    输出：'你好，能帮个忙吗？'

    - 输入：'怎么重写句子结构？'
    输出：'句子结构优化步骤示例'

输出时无需解释，仅返回最终结果。
"""


async def fetch_search_results(url, headers, payload):
    """发动请求并获取响应"""
    async with aiohttp.ClientSession() as session:
        async with session.post(url, headers=headers, json=payload) as response:
            assert response.status == 200
            response_json = await response.json()
            return response_json  # 返回响应json
```

```python

async def query_rewrite(query:str):
    """一个简单的query改写，直接用aiohttp了，没有使用openai"""
    url = chat_base_url
    headers = {
                'Authorization': f'Bearer {reranking_api_key}',
                'Content-Type': 'application/json'
                }
    payload={
                "model": "Qwen/Qwen2.5-72B-Instruct",
                "messages": [
                                {
                                "role": "system",
                                "content": query_rewrite_prompt
                                },
                                {
                                "role": "user",
                                    "content": query
                                }
                            ],
                "max_tokens": 512,
                "stream":False
    }
    response_data = await fetch_search_results( url=url,
                                                headers=headers,
                                                payload=payload
                                                )
    rewrite_query = response_data['choices'][0]['message']['content']
    return rewrite_query


async def search_with_serper(query:str,document_num:int=10):
    """google serper 搜索并返回documents,形式如下

    [{"title1":"","link1":"","snippet1":"","position1":""},
     {"title2":"","link2":"","snippet2":"","position2":""},
     ...
    ]

    """
    url = google_serper_url
    headers = {
        'X-API-KEY': google_serper_api,
        'Content-Type': 'application/json'
    }
    payload = {
        "q": query, # 要搜索的问题
        "num": document_num # 要所搜的文档数量
    }


```

```
    response_data = await fetch_search_results( url=url,
                                                headers=headers,
                                                payload=payload
                                                )

    documents = response_data['organic']
    return documents


async def reranking_by_bge(query:str,documents:list[dict],top_n:int=4):
    """调用排序模型，返回排序后的文档"""
    doc_to_rerank = [doc["snippet"] for doc in documents]

    url = reranking_base_url
    headers = {
                'Authorization': f'Bearer {reranking_api_key}',
                'Content-Type': 'application/json'
              }
    payload = {
                "model": reranking_model,
                "query": query,
                "documents": doc_to_rerank,
                "top_n": top_n,
                "return_documents": True, # 如果是false则只返回索引，麻烦
                "max_chunks_per_doc": 1024,
                "overlap_tokens": 80
            }

    reranked_info = await fetch_search_results( url=url,
                                                  headers=headers,
                                                  payload=payload
                                                )
    reranked_doc =[item['document']['text'] for item in reranked_info['results']]

    return reranked_doc


async def test():
    import rich
    query = "流水不争先"
    rewrite_query = await query_rewrite(query=query)
    rich.print('rewrite_query',rewrite_query)
    documents = await search_with_serper(query=rewrite_query,document_num=4)
    rich.print('documents',documents)
    reranked_doc = await reranking_by_bge(query=query,documents=documents,top_n=2)
    rich.print("reranked_doc",reranked_doc)

```

```python

if __name__ == "__main__":
    asyncio.run(test())

```

test函数是用来测试的

```
(rag_sample) PS C:\Users\孟智超\Desktop\mac_backups\MCP\rag_sample> python .\utils.py
rewrite_query 流水不争先，何意？
documents
[
    {
        'title': '流水不争先争的是什么意思 - 起点中文',
        'link': 'https://m.qidian.com/ask/qbpbpnjurtu',
        'snippet':
'“流水不争先”源出《道德经》，意思是做事情要顺其自然，脚踏实地，不争一时之短长，就像流水一样，不会争先奔涌，只是缓缓流淌，积蓄力量，以待
迸发。不争先并非不进取，而是要尊重 ...',
        'position': 1
    },
    {
        'title': '流水不争先，争的是滔滔不绝。到底是什么意思呢？ - 知乎',
        'link': 'https://www.zhihu.com/question/353079553',
        'snippet': '流水不争先，出自老子道德经，水善万利而不争， 更多的是老子的一种无为，不争的思想.
也是国立老师所演出的话剧断金里面的富小莲的一句台词，. 在语境里.',
        'date': 'Oct 29, 2019',
        'position': 2
    },
    {
        'title': '《道德经》：流水不争先，争的是滔滔不绝 - 搜狐',
        'link': 'https://www.sohu.com/a/206686516_523098',
        'snippet': '“流水不争先”的智慧来源于老子的《道德经》。
《道德经》中，“水善利万物而不争”“不敢为天下先”“为而不争”，等等，说的都是“不争先”的智慧。 天道自行循环运 ...',
        'date': 'Nov 26, 2017',
        'position': 3
    },
    {
        'title': '流水不争先，争的是滔滔不绝---写给高考成绩不理想的同学们 - 学术桥',
        'link': 'https://www.acabridge.cn/hr/xueshu/202406/t20240625_2619087.shtml',
        'snippet':
'流水不争先，因为它懂得尊重自然规律，不去强行改变事物的节奏和进程。同样地，我们也要学会尊重自己成长的节奏，不要因为一时的落后而自乱阵脚
。每个人都有 ...',
        'date': 'Jun 25, 2024',
        'position': 4
    }
]
reranked_doc
[
    '流水不争先，出自老子道德经，水善万利而不争， 更多的是老子的一种无为，不争的思想.
也是国立老师所演出的话剧断金里面的富小莲的一句台词，. 在语境里.',
    '“流水不争先”源出《道德经》，意思是做事情要顺其自然，脚踏实地，不争一时之短长，就像流水一样，不会争先奔涌，只是缓缓流淌，积蓄力量，
以待迸发。不争先并非不进取，而是要尊重 ...'
]
(rag_sample) PS C:\Users\孟智超\Desktop\mac_backups\MCP\rag_sample>
```

- 运行 python rag_client.py以确认一切正常

2.2.3 app端使用(cherrystudio为例)

- 安装cherrystudio,并配置好模型

> https://cherry-ai.com/

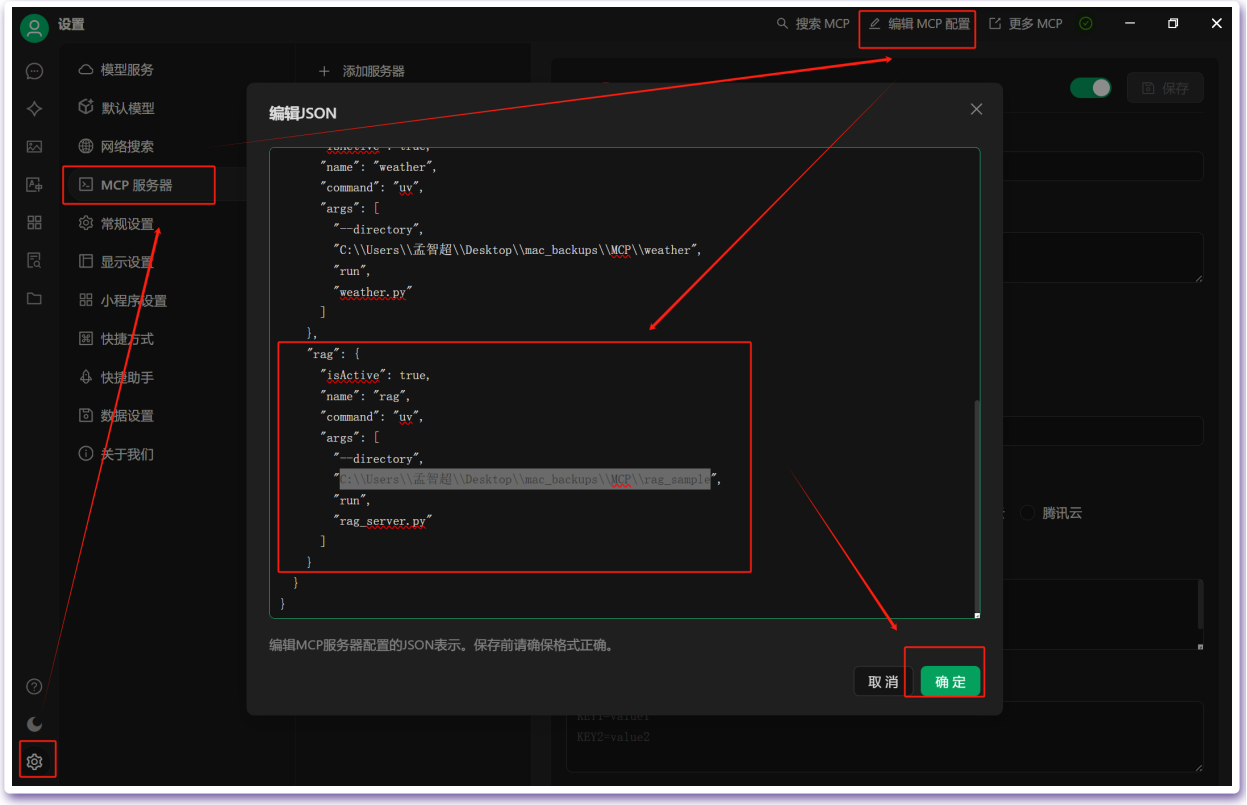
- 在app端添加单个weather服务器,下面的json文件需要复制到cherrystudio中

```
{
    "mcpServers": {
        "rag": {
            "command": "uv",
            "args": [
                "--directory",
                "C:\\Users\\孟智超\\Desktop\\mac_backups\\MCP\\rag_sample",
                "run",
                "rag_client.py"
            ]
        }
    }
}
```

意思是：

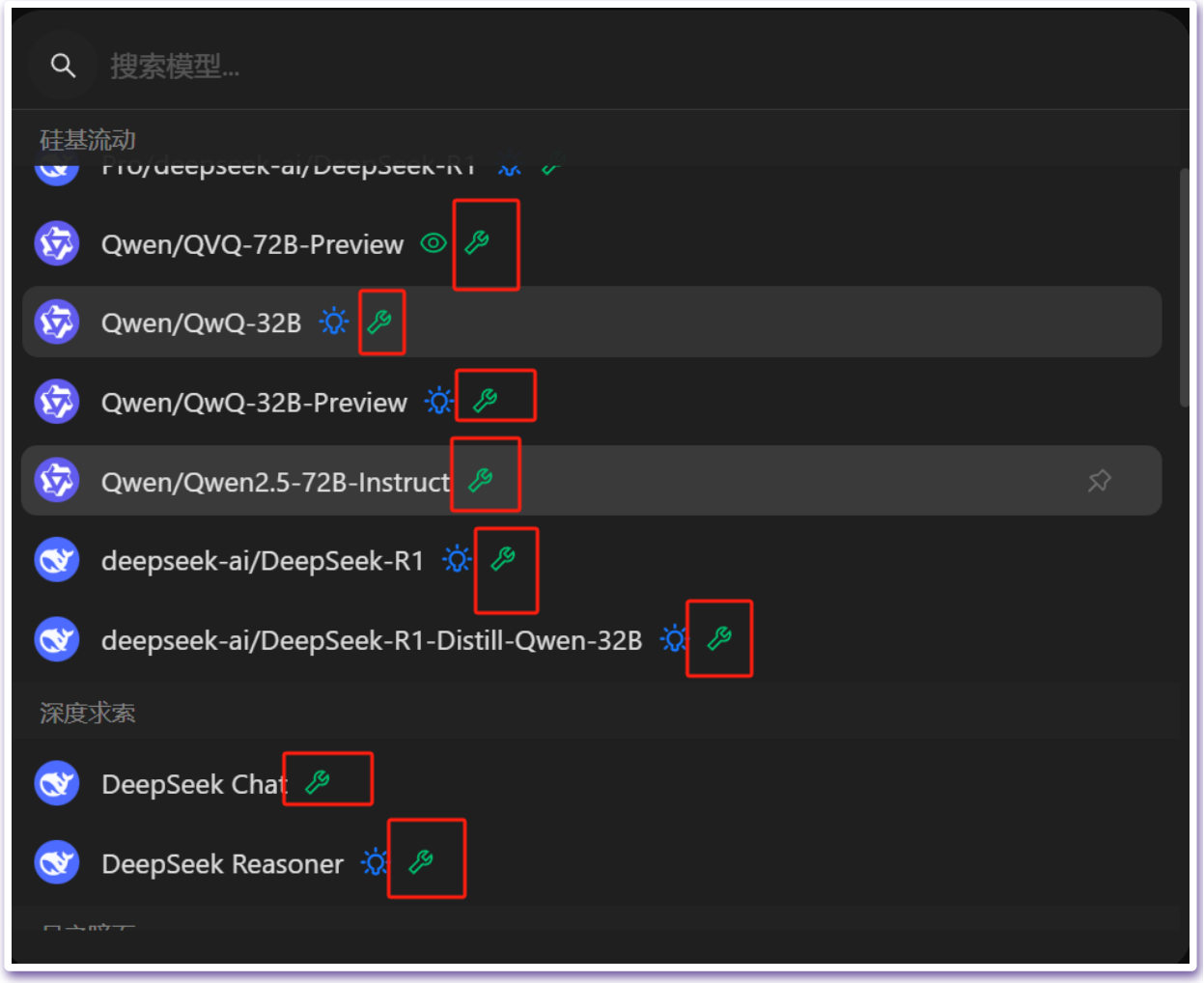
1. 有一个名为“天气”的 MCP 服务器
2. 通过运行 `uv --directory C:\\Users\\孟智超\\Desktop\\mac_backups\\MCP\\rag_sample run rag_client.py` 来启动它（项目的地址）

- 将上面的json文本复制到cherrystudio

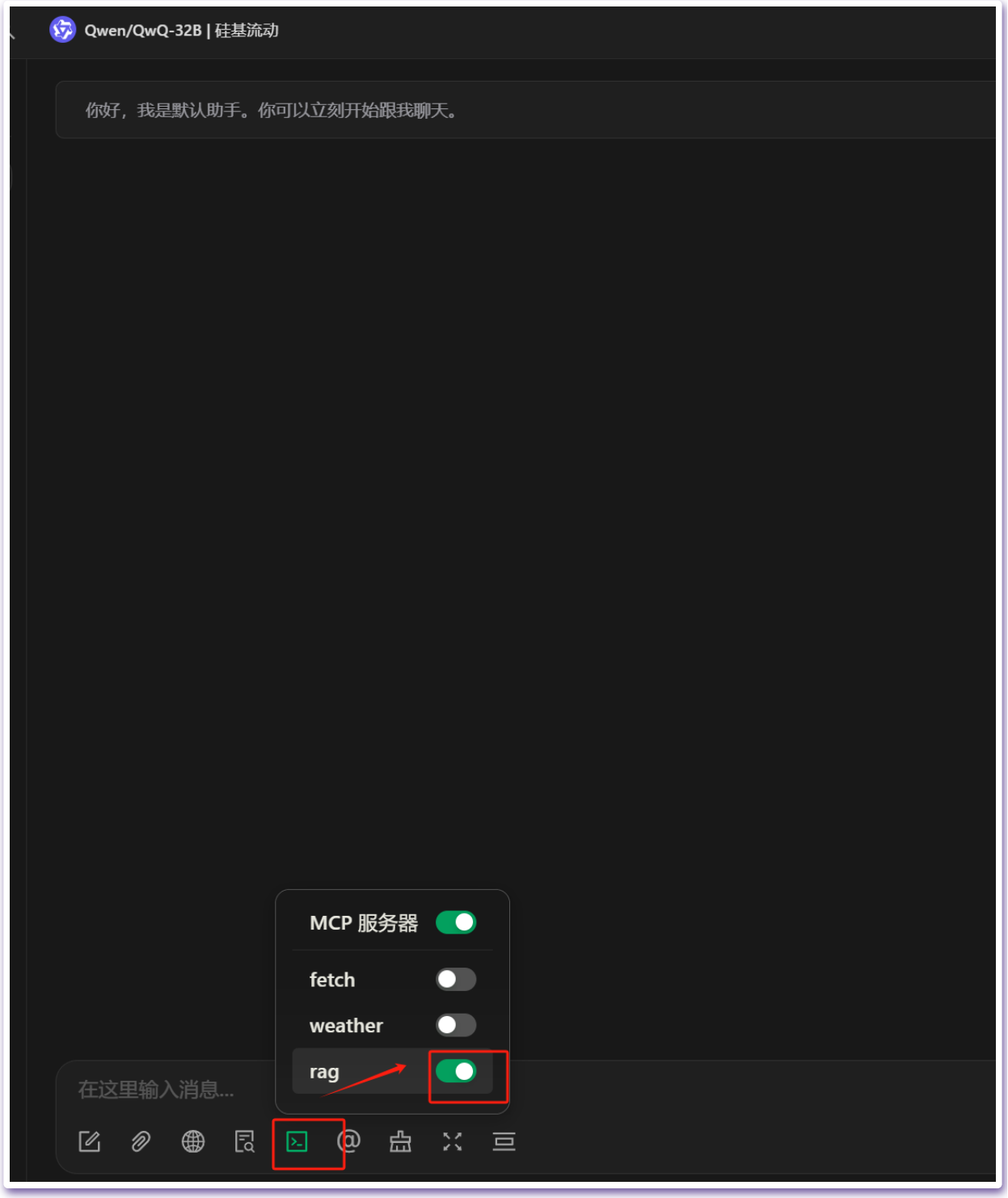


- 选择模型，因为mcp也借用了functioncalling，所以需要选用的模型必须支持functioncalling（tools）

在cherrystudio中的表现就是，有绿色的小扳手



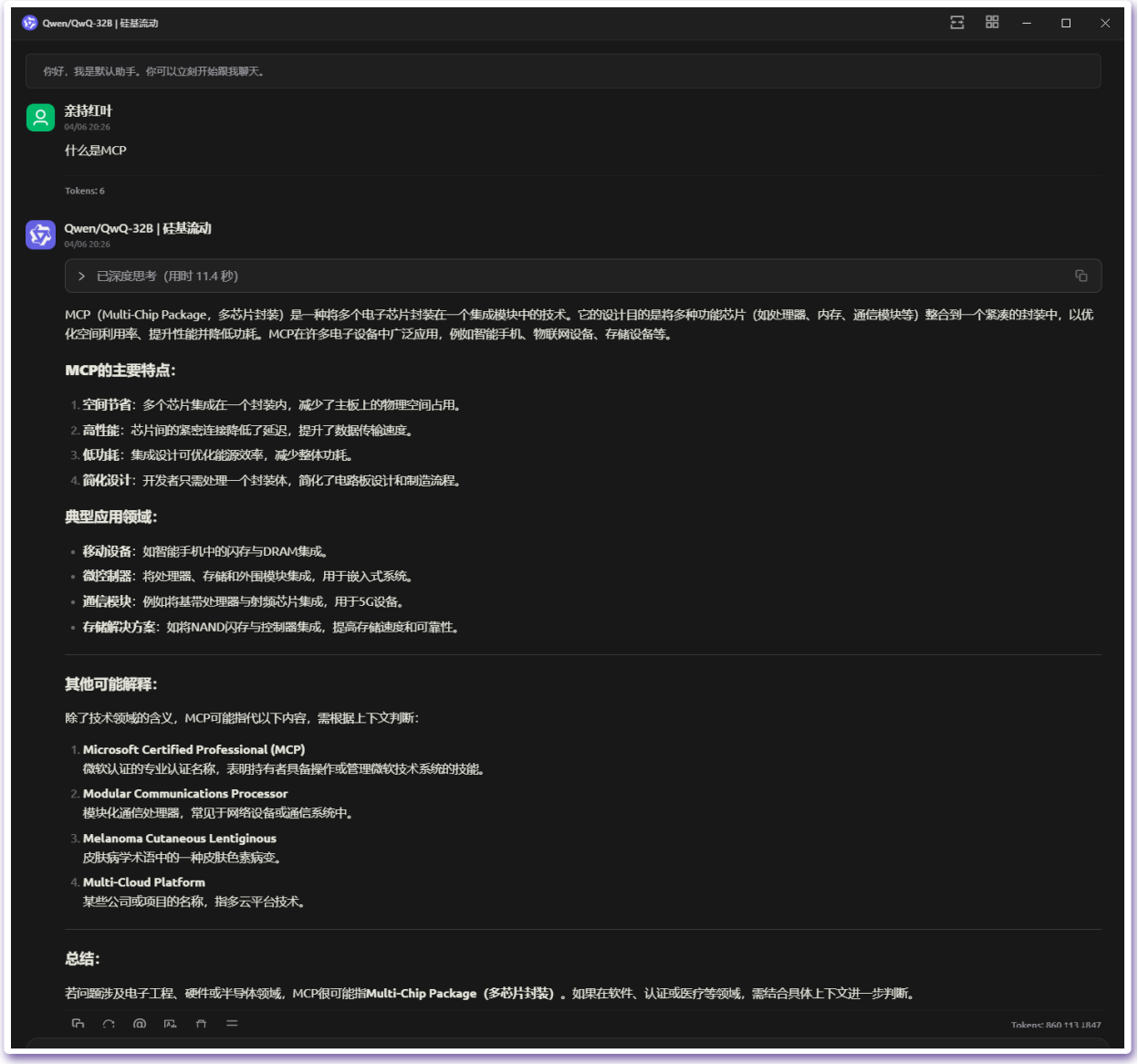
- 开始聊天之前开启mcp



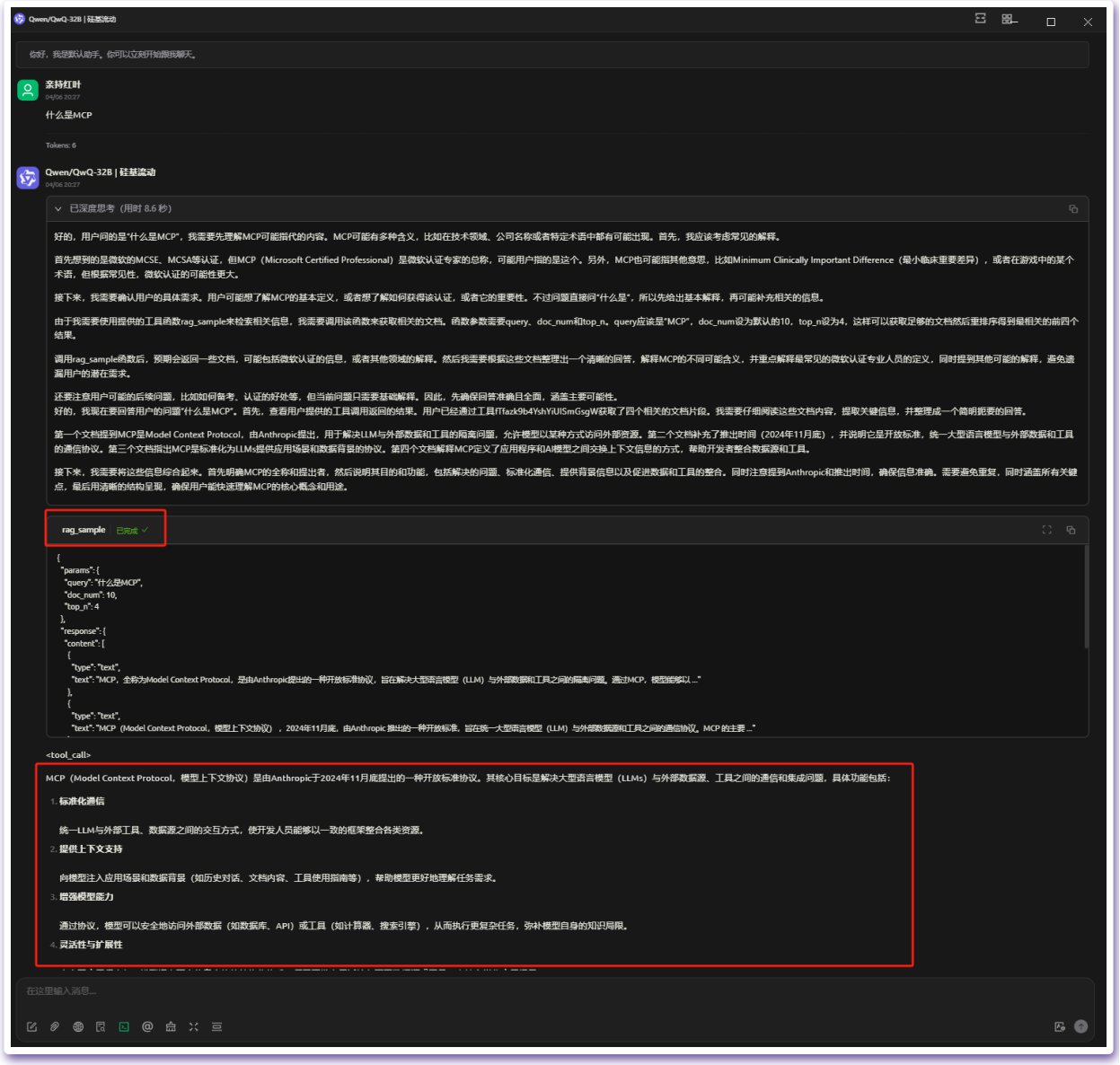
- 看看使用前后的对比

  问题：什么是MCP?

  未开启前,列举了很多，但是都不是我们想要的



开启后，虽然在思考过程中也提到了其他无关的，但是最后还是决定调用工具 rag_sample，最后得到的是我们想要的正确答案。



2.3 客户端

2.3.1 代码

- 和服务器共用一个环境（不推荐，一般重新开一个环境）
- 代码 `rag_client.py`

```
import os
import sys
import json
import asyncio
from typing import Optional
from contextlib import AsyncExitStack

from mcp import ClientSession,StdioServerParameters
from mcp.client.stdio import stdio_client

```

```python
import openai
from openai import OpenAI
from dotenv import load_dotenv
import traceback

# 从.env文件加载环境变量
load_dotenv()

# siliconflow api
openai.base_url = os.getenv("OPENAI_BASE_URL")
openai.api_key = os.getenv("SILICONFLOW_RERANKING_API_KEY")
openai_chat_model = os.getenv("OPENAI_CHAT_MODEL")

class MCPClient:
    def __init__(self):
        # 初始化会话和客户端对象
        self.session: Optional[ClientSession] = None # 保存mcp客户端会话
        self.exit_stack = AsyncExitStack()
        self.model = openai_chat_model
        self.openai = OpenAI(
            base_url=openai.base_url,
            api_key=openai.api_key
        )

    async def connect_to_server(self, server_script_path:str):
        """connect_to_server:   连接到mcp服务器


        Parameters:
        ------------------------------------------------------------------------------------------------------------
        server_script_path:         type:str             desc: server脚本地址（.py or .js）
        """
        is_python = server_script_path.endswith('.py') # python脚本?
        is_js = server_script_path.endswith('.js') # node 脚本?

        if not (is_python or is_js):
            raise ValueError("Server script must be a .py or .js file")

        command = "python" if is_python else "node"
        server_params = StdioServerParameters(
            command=command,
            args=[server_script_path],
            env=None
        ) # 告诉mcp客户端如何启动服务器,比如当前的 python rag_server.py

        stdio_transport = await self.exit_stack.enter_async_context(
```

```
            stdio_client(server_params) # 启动服务器进程，建立标准I/O通信管道
            )
        self.stdio,self.write = stdio_transport # 拿到读写流
        self.session = await self.exit_stack.enter_async_context(
            ClientSession(self.stdio,self.write) # 创建MCP客户端会话，与服务器交互
            )

        await self.session.initialize() # 发送初始化消息给服务器，等待服务器就绪。

        # 列出 mcp 服务器上可用的工具
        response = await self.session.list_tools() # 向 mcp 服务器请求所有已注册的工具（即@mcp.tool）
        tools = response.tools
        print("\nConnected to server with tools:", [tool.name for tool in tools])

    async def process_query(self,query:str) -> str:
        """process_query:   使用大模型处理查询，调用已经注册的MCP工具


        Parameters:
        ------------------------------------------------------------------------------------------------------------
        query:                type:str                desc:用户问题

        """
        messages = [
            {
                "role":"user",
                "content": query
            }
        ]

        response = await self.session.list_tools() # 向 mcp 服务器请求所有已注册的工具（即@mcp.tool）
        # 转换为functional calling 的available_tools格式
        available_tools = [{
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description,
                "input_schema": tool.inputSchema
            }} for tool in response.tools]
        # print(f"available_tools: \n{available_tools}")

        # 初始化 大模型连接(OpenAI)
        response = self.openai.chat.completions.create(
            model=self.model,
            max_tokens=1000,
```

```
            messages=messages,
            tools=available_tools
        )
        content = response.choices[0]
        print(f"content.finish_reason : {content.finish_reason}")
        if content.finish_reason == "tool_calls":
            print(f"进入functional calling")

            # content.finish_reason == "tool_calls"表示要调用工具
            # 会在content.message.tool_calls 列表中声明要用哪个函数、参数是什么
            tool_call = content.message.tool_calls[0]
            tool_name = tool_call.function.name
            tool_args = json.loads(tool_call.function.arguments)

            # 取出工具名 tool_name 和参数 tool_args，执行mcp工具
            result = await self.session.call_tool(tool_name, tool_args)
            print(f"\n[Calling tool {tool_name} with args {tool_args}]\n\n")
            # print(f"\n\ncall_tool_result: {result}")
            # print(f"\n\ncall_tool_result.content[0].text: {result.content[0].text}")
            # print(f"i.text for i in result.content:{[i.text for i in result.content]}")

            # 将模型返回的调用哪个工具数据和工具执行完成后的数据都存入messages中
            messages.append(content.message.model_dump())
            messages.append({
                "role": "tool",
                "content": f"{[i.text for i in result.content]}",
                "tool_call_id": tool_call.id,
            })

            # 将上面的结果再次返回给大模型并产生最终结果
            response = self.openai.chat.completions.create(
                model=self.model,
                max_tokens=1000,
                messages=messages,
            )
            return response.choices[0].message.content

        # 如果没有调用工具则直接返回当前结果
        return content.message.content

    async def chat_loop(self):
        """运行一个交互式聊天循环"""
        print("MCP 服务器已经启动!")
        print("输入你的问题（输入'quit'退出）.")

        while True:
```

```python
            try:
                query = input("\n你的问题:\n").strip()
                if query.lower() == "quit":
                    break

                response = await self.process_query(query)
                print(f"模型回答: \n{response}")

            except Exception as e:
                tb = traceback.extract_tb(e.__traceback__)
                for frame in tb:
                    print("File:", frame.filename)
                    print("Line:", frame.lineno)
                    print("Function:", frame.name)
                    print("Code:", frame.line)

    async def cleanup(self):
        """清理资源"""
        # 异步地关闭所有在 exit_stack 中注册的资源（包括 MCP 会话）。
        await self.exit_stack.aclose()

async def main():
    """main:   主执行逻辑，以现在的rag为例子
    标准IO启动需要服务器的代码作为客户端的子进程启动
    服务器代码: rag_server.py
    客户端代码: rag_client.py
    因为都在同一个项目下，所以启动直接为
    python rag_client.py rag_server.py

    """
    if len(sys.argv) < 2: #
        print("Usage: python client.py <path_to_server_script>")
        sys.exit(1)

    client = MCPClient()
    try:
        print(sys.argv[1])
        # python rag_client.py rag_server.py ,则argv[1] = rag_client.py 为客户端代码
        await client.connect_to_server(sys.argv[1])
        await client.chat_loop()
    finally:
        await client.cleanup()

if __name__ == "__main__":
    asyncio.run(main())
```

1. `connect_to_server`： 连接到MCP服务器
2. `process_query`： 使用大模型处理查询，将MCP工具转换为OpenAI兼容的格式，允许模型决定是否调用工具，如果是，则调用已经注册的MCP工具
3. `chat_loop`： 交互式聊天（只有单轮）

4. `main`：主函数

   因为在客户端脚本中，启动方式是"stdio"

   ```
   if __name__ == "__main__":
       mcp.run(transport="stdio")
   ```

   这种适合在客户端和服务器在同一个环境中的情况下，是将服务器作为客户端的子进程

   所以其中这个客户端的命令如下

   ```
   python rag_client.py rag_server.py
   ```

2.3.2 测试运行

- 启动

  ```
  python rag_client.py rag_server.py
  ```

- 测试下

  (模型不太好触发这个tool，所以我直接加了“请调用rag_sample”)

  问题：

  ```
  请调用rag_sample,解释下最近很火的mcp
  ```

```
(rag_sample) PS C:\Users\孟智超\Desktop\mac_backups\MCP\rag_sample> python .\rag_server.py
[
    'MCP协议是什么？MCP(Model Context Protocol，模型上下文协议)
是一种新的开放标准协议，用来在大模型和数据源之间建立安全双向的链接。对照下面这个图，我们来理解下MCP。',
    'Model Context Protocol (MCP)
模型上下文协议，是由claude的发布公司Anthropic，提出的一种开放协议。为给LLM提供上下文（数据）提供了一种标准协议。',
    'MCP（Model Context Protocol，模型上下文协议）是由Anthropic 提出并于2024 年11
月开源的一种通信协议，旨在解决大型语言模型（LLM）与外部数据源及工具之间无 ...',
    'MCP 是一种新的开放标准协议，用来在大模型和数据源之间建立安全双向的链接。这是它的官方示意图。 这张图展示了使用Claude 或其他IDE
时，通过这种 ...'
]
(rag_sample) PS C:\Users\孟智超\Desktop\mac_backups\MCP\rag_sample> python rag_client.py rag_server.py
rag_server.py

Connected to server with tools: ['rag_sample']
MCP 服务器已经启动！
输入你的问题（输入'quit'退出）.

你的问题：
你好
content.finish_reason : stop
模型回答：
你好！有什么可以帮你的吗？

你的问题：
请调用rag_sample,解释下最近很火的mcp
content.finish_reason : tool_calls
进入functional calling

[Calling tool rag_sample with args {'query': '解释最近很火的MCP模型', 'doc_num': 5, 'top_n': 3}]

模型回答：

MCP（Model Context Protocol，模型上下文协议）是近期在AI领域受到广泛关注的新技术框架，它旨在解决AI模型在实际应用中的一个核心问题：**如何
让模型动态利用外部实时数据或工具**，从而提升其解决复杂任务的能力。以下是基于文档信息的详细解释：

---

### **1. MCP的核心概念**
- **定义**：MCP是一种标准化协议，定义了AI模型与外部数据源、工具或应用程序之间交互的规则和接口。
- **目标**：
  通过统一的接口规范，让模型能够**动态调用外部资源**（如实时数据库、API、传感器数据、第三方工具等），而不再依赖于训练时固定的静态数据。
这使得模型在推理过程中能“主动”获取当前需要的信息，解决实际问题。

---

### **2. 为什么需要MCP？**
传统AI模型（如大语言模型）存在一个显著限制：它们的知识被冻结在其训练时的数据截止日期之前，无法访问实时数据或外部工具。例如：
- **旅行规划问题**：模型可能无法获取最新航班信息、实时天气变化或铁路时刻表，导致建议不准确。
- **医疗诊断**：模型无法调用患者最新检查结果或实时医学指南。
- **金融分析**：模型无法访问实时股市数据或经济指标。

MCP的出现直接解决了这些问题，让模型能够像人类一样，根据任务需求**即时调用外部数据或工具**，从而生成更精准、实用的输出。

---

### **3. MCP的主要功能与特点**
1. **动态上下文整合**
   模型在推理时，可以主动调用外部API、数据库或工具，获取最新的数据或执行特定操作（如订票、查询天气）。
   - **例如**：在旅行规划场景中，MCP允许模型调用航班API获取实时机票价格、

你的问题：
```

第一轮没有调用工具

第二轮调用了工具，并传入上述参数